CESI École d'ingénieurs et SEMCO Technologies

Architectures de consolidation de données : Analyse comparative entre ETL et Fédération de requêtes

Auteur :
Trystan JULIEN

15 mars 2026

Résumé

SEMCO Technologies est un acteur mondial incontournable dans les composants stratégiques pour la fabrication des semi-conducteurs de nouvelle génération. Sa production doit donc refléter son rôle dans cette industrie. C'est dans ce but qu'une nouvelle ligne de production automatisée est en cours de déploiement, permettant une modernisation et une augmentation progressive des capacités de la production de l'entreprise.

Dans ce contexte, la gestion et la centralisation des données issues de cette ligne automatisée constituent un enjeu important pour l'exploitation et l'analyse des informations de production.

Ce travail a pour objectif de documenter une comparaison pratique entre différentes architectures de consolidation des données. L'étude vise à déterminer l'approche la plus adaptée pour centraliser les données générées par la ligne de production automatisée vers la base de données principale alimentant la solution logicielle SP-Manager, utilisée au sein de l'entreprise pour l'exploitation et la gestion des données de production.

Pour ce faire, plusieurs approches d'intégration des données seront étudiées et comparées au moyen de tests de performance et d'analyses techniques, afin d'identifier la solution la plus pertinente dans le contexte de l'infrastructure existante.

Abstract

SEMCO Technologies is a key global player in strategic components for the manufacturing of next-generation semiconductors. Its production must reflect its role in this industry. With this goal in mind, a new automated production line is being deployed, allowing the modernization and the progressive increase of the company's production capacity.

With this context, the centralisation and handling of data gathered from this automated line becomes a key challenge regarding the exploitation and analysis of production information.

This work aims to provide practical comparison between different architectures of data consolidation. This study's goal is to determine the most suitable approach in order to centralise the data generated by the automated production line in the main database providing the software SPManager. Which is used inside the company for the exploitation and handling of production data.

To reach those objectives, multiple approaches of data integration will be studied and compared using performance tests and a technical analysis, in order to identify the most relevant solution in the context of the existing architecture.

Table des matières

Glossaire **V**

1 Introduction **1**

2 Contexte Technique **2**
2.1 Les bases de données . . . 2
2.1.1 Base principale . . . 2
2.1.2 Base secondaire . . . 3
2.2 Architectures actuelles . . . 4
2.2.1 Architecture globale . . . 4
2.2.2 Flux de données . . . 4
2.3 Stack technique . . . 5

3 État de l'art **6**
3.1 L'architecture fédérée . . . 6
3.1.1 La fédération par vue virtuelle . . . 6
3.1.2 La fédération par hub central . . . 6
3.2 ETL . . . 7
3.2.1 Extract . . . 7
3.2.2 Transform . . . 7
3.2.3 Load . . . 7
3.3 Jointures distribuées . . . 8
3.4 Centralisation des données . . . 9
3.4.1 Avantages . . . 9
3.4.2 Inconvénients . . . 9
3.5 Problèmes de performances . . . 10
3.5.1 Coût des communications réseau . . . 10
3.5.2 Coût des jointures distribuées . . . 10
3.5.3 Difficulté d'optimisation des requêtes . . . 10
3.5.4 Déséquilibre des données . . . 10

4 Architecture des solutions **11**
4.0.1 Architecture ETL . . . 11
4.0.2 Architecture Fédérée . . . 12
4.0.3 Synthèse de chapitre . . . 12

5 Méthodologie de test **13**
5.1 Introduction . . . 13
5.2 Environnement de test . . . 13
5.2.1 Infrastructure matérielle et logicielle . . . 13
5.2.2 Implémentation de l'architecture ETL . . . 14
5.2.3 Implémentation de l'architecture fédérée . . . 14
5.3 Jeux de données . . . 15
5.3.1 Choix du benchmark . . . 15
5.3.2 Distribution des données entre les bases . . . 15
5.4 Scénarios de requêtes . . . 16
5.5 Métriques mesurées . . . 16
5.5.1 Métriques de performance . . . 16
5.5.2 Protocole de mesure et traitement statistique . . . 17
5.6 Limites et biais de la méthodologie . . . 17

6 Résultats des tests et Analyse : 18
6.1 Condition d'exécution . . . 18
6.2 T1 - Lecture simple . . . 19
6.3 T2 & T3 - Requêtes complexes . . . 21
6.3.1 T2 - Jointure complexe . . . 21
6.3.2 T3 - CTE avec fenêtrage + LAG . . . 23
6.3.3 Synthèse T2 + T3 . . . 24
6.4 T4 - Écriture unitaire . . . 25
6.5 T5 & T7 - Comportement sous charge . . . 26
6.5.1 T5 - SELECT avec jointures en rafale . . . 26
6.6 T7 - INSERT simple en rafale . . . 29
6.7 T6 - Ingestion batch . . . 31
6.8 Synthèse de l'étude . . . 32

7 Conclusion générale et ouverture 33

Bibliographie 34

Liste des figures

2.1 Schéma de l'architecture globale du système d'information de la gestion de production 4
2.2 Schéma du flux de données du système SPManager 4
2.3 Schéma du flux de données de la ligne automatique 5

4.1 Schéma de l'architecture ETL 11
4.2 Schéma de l'architecture fédérée 12

6.1 Graphique du temps d'exécution end-to-end de la requête T1 19
6.2 Graphique des lectures logiques de la requête T1 20
6.3 Graphique de la charge CPU pour T2 21
6.4 Graphique des lectures logiques pour T2 22
6.5 Graphique des lectures logiques pour T3 23
6.6 Graphique du temps d'exécution end-to-end et serveur de T4 25
6.7 Graphiques des débits de T5 26
6.8 Graphiques des temps de complétions de T5 27
6.9 Graphique de la consommation mémoire .NET de T5 28
6.10 Graphiques des débits de T7 29
6.11 Graphiques des temps de complétion end-to-end de T7 29
6.12 Graphiques de la mémoire SQL de T7 30
6.13 Tableau de l'ingestion ETL durant T6 31
6.14 Graphique de l'ingestion ETL durant T6 31

Liste des tableaux

2.1 Résumé des caractéristiques de la base principale 2
2.2 Résumé des caractéristiques de la base secondaire 3

5.1 Configuration de l'environnement de test 13
5.2 Facteurs d'échelle retenus pour les tests 15
5.3 Scénarios de test du benchmark 16
5.4 Récapitulatif des métriques par scénario 17

Glossaire :

- **Architecture fédérée** = Architecture dans laquelle des sources de données séparées physiquement et hétérogène sont centralisé en dans un stockage central.
- **BDD** = Base De Données, sytème qui permet un accès facile à un ensemble organisé de données.
- **Bruit de mesure** = Informations parasites venant se superposer sur la mesure que l'on cherche à obtenir.
- **Buffer pool** = Zone de la mémoire principale dans laquelle sont mise en cache les données de table et index telles qu'elles sont lues depuis le disque.
- **Cardinalité** = Nombre de lignes présente dans une table ou retourner par une requête.
- **Contention CPU** = Situation où plusieurs processus veulent accéder aux ressources du processeur en même temps, créant des blocages et files d'attentes.
- **Contention d'écriture** = Situation où une requête demande une écriture ou modification sur les mêmes pages de données ou index qu'une autre requête, créant une attente.
- **CPU** = Central Processing Unit
- **CRM** = Customer Relationship Management, Système de gestion des relations clients. Il permet de centraliser et d'exploiter les informations en lien avec les clients et les interactions commerciales d'une entreprise.
- **CTE** = Common Table Expression, Ensemble de résultats temporaire nommé et créé dans une requête principale à partir d'une sous-requête SQL, dont les résultats sont exploitables dans cette même requête.
- **Dapper** = Mappeur d'objet relationnel léger pour .NET. Il permet de transformer les objets .NET en leur équivalent dans la base de données relationnelle et inversement, tout en conservant l'accès direct aux requêtes SQL.
- **Data skew** = Situation dans laquelle les données sont inégalement partitionnées sur les différents nœuds d'un système distribué.
- **Data Warehouse** = Base de données relationnelle centralisant les données venant de sources distantes et hétérogènes, outil principal de l'analyse et de l'aide à la décision.
- **DBCC** = DataBase Console Command
- **DBCC DROPCLEANBUFFERS** = Commande SQL Server servant à vider manuellement le cache du buffer pool, utilisée dans chaque série de test pour garantir des conditions initiales identiques.
- **DDR** = Double Data Rate
- **Déduplication** = Procédé d'identification des données redondantes ou en double au sein d'un ensemble de données, permettant de garantir l'unicité des informations stockées.
- **DMV** = Dynamic Management View, Ensemble de requêtes permettant de récupérer des informations sur l'état du serveur, les performances et l'activité en cours au sein d'une base de données.
- **ERP** = Enterprise Resource Planning
- **ETL** = Extract-Transform-Load, Processus consistant à récupérer les données d'une source externe, les transformer pour les rendre exploitable pour l'analyse et l'insérer dans une base de données centralisée.
- **Fenêtrage (Window Function)** = Fonction qui établit un sous-ensemble de lignes défini par une partition et un ordre, permettant des opérations sur cet ensemble sans en réduire le nombre de lignes.
- **GC** = Garbage Collector, Sous-système de gestion automatique de la mémoire, qui libère la mémoire précédemment allouée et maintenant inutilisée.
- **GHz** = GigaHertz
- **HDD** = Hard Disk Drive
- **I/O** = Input/Output
- **IQR** = Interquartile Range (Écart Inter-quartile), Mesure de la dispersion des données autour de la médiane permettant entre autre d'observer la stabilité d'une mesure.

- **Jointure distribuée** = Jointures entre tables se trouvant sur des bases de données distinctes et nécessitant le transfert des données entre les sources avant de pouvoir produire un résultat.
- **LAG** = Fonction de fenêtrage SQL, permet d'accéder aux valeurs d'une ligne différente se trouvant au-dessus de la ligne actuelle.
- **Linked Server** = Création d'un lien permettant au moteur de gestion de base de données d'exécuter des commandes sur des sources de données se situant en dehors de l'instance.
- **LINQ** = Language-INtegrated Query, composant du framework .NET permettant d'interroger des données dans le code avec un langage similaire à SQL.
- **Loopback TCP** = Interface réseau virtuelle qui permet à deux processus de communiquer entre eux sur une même machine via le protocole TCP/IP, simulant une connexion réseau complète sans passer par un réseau physique externe.
- **Médiane** = Valeur dans un ensemble de données qui, lorsque rangées dans l'ordre croissant, sépare l'ensemble en deux sous-ensembles de même effectif.
- **Mémoire managée .NET** = Zone de mémoire allouée et gérée automatiquement par le runtime .NET via le garbage collector, libérant le développeur de la gestion manuelle de la mémoire.
- **Mo/Go** = Mégaoctet / Gigaoctet
- **MS** = MilliSeconde
- **Nœud (système distribué)** = Défini un ordinateur ou un serveur individuel faisant partie d'un système distribué.
- **NVMe** = NonVolatile Memory Express
- **ORM** = Object-Relational Mapping, Type de programme informatique permettant de traduire les résultats d'une requête SQL en objets et inversement.
- **OS** = Operating System
- **Pages sales (Dirty pages)** = Page de données SQL mise à jour en mémoire et attendant sa réécriture sur le disque.
- **Pause stop-the-world** = Mécanisme de mise en pause utilisé par le Garbage Collector, suspendant l'exécution de tous les threads actifs le temps de libérer la mémoire inutilisée.
- **Plan d'exécution** = Plan que le système de gestion de base de données choisit pour répondre à une requête de la manière la plus optimale.
- **Prédicat (jointure)** = Condition utilisée pour définir le lien logique entre deux tables, plus simplement quelles lignes seront liées entre elles.
- **Quartz.NET** = Bibliothèque .NET permettant la planification de tâches, donc l'exécution automatique de processus selon un planning défini.
- **RAM** = Random Access Memory
- **Scheduler Windows** = Composant du noyau système qui décide quel processus a accès aux cœurs du processeur, quand et pendant combien de temps ils ont accès.
- **Sérialisation TDS** = Processus convertissant les données au format TDS.
- **SGBD** = Système de Gestion de Base de Données
- **SGBDR** = Système de Gestion de Base de Données Relationnelles
- **SQL** = Structured Query Language
- **SqlBulkCopy** = Fonction permettant l'insertion massive de données dans une table SQLServer de manière plus optimisée qu'avec un INSERT SQL classique.
- **SSD** = Solid State Drive
- **TCP** = Transmission Control Protocol, Norme de transmission de données et de message via le réseau.
- **TDS** = Tabular Data Stream, Protocole applicatif de SQL Server qui permet la transmission des données entre le serveur et le client
- **Thread pool .NET** = Ensemble de threads préexistant et prêt à exécuter des tâches, permettant d'éviter les surcharges liées à la création et suppression des threads.
- **Warm-up** = Première exécution d'un programme dans le but de préparer les composants importants afin que leur initialisation n'affecte pas les mesures.
- **Windows Service .NET** = Programme .NET qui s'exécute en arrière-plan sur Windows sans interface utilisateur. Il démarre automatiquement avec le système et permet l'exécution continue ou périodique de tâches.

- **WPF** = Windows Presentation Foundation, Spécification graphique de Microsoft utilisant le langage graphique XAML, permettant une expérience de développement de l'interface utilisateur plus proche du HTML pour les développeurs.
- **XMP** = eXtreme Memory Profile

Chapitre 1

Introduction

La fabrication de semi-conducteurs est un secteur technologiquement exigeant. La qualité et la précision des mesures ainsi que la rapidité de production sont des facteurs extrêmement importants. De ce fait, les entreprises cherchent à mettre en place de nouvelles solutions d'automatisation avancées, notamment pour la collecte et l'analyse de données de production.

SEMCO Technologies, en tant qu'acteur international dans ce secteur, modernise sa production de la même manière grâce à la mise en place d'une nouvelle ligne automatisée. Cet ajout s'accompagne de nouvelles données de production qu'il faut récupérer et centraliser.

Le cadre technique actuel repose sur deux bases de données distinctes. La première correspond à la base de données principale exploitée par la solution logicielle SPManager, dont l'objectif est de centraliser l'ensemble des données de production afin de permettre leur exploitation et leur analyse. La seconde base de données stocke uniquement les données générées par la nouvelle ligne automatisée et n'est actuellement reliée ni à la base principale ni à l'application SPManager.

Cette situation met en évidence un besoin de centralisation des données. Plusieurs approches peuvent être envisagées pour répondre à cette problématique. Parmi celles-ci, deux architectures seront étudiées dans ce travail : une approche fédérée, dans laquelle les données sont interrogées directement depuis leurs sources respectives, et une approche reposant sur un processus ETL permettant de consolider les données dans une base centrale.

L'objectif de ce travail est de déterminer la solution la plus efficace et la plus performante pour réaliser cette centralisation.

La question de recherche peut ainsi être formulée de la manière suivante :

> Quelle architecture de consolidation des données permet de centraliser efficacement les informations issues de la ligne automatisée tout en garantissant de bonnes performances d'accès ?

Dans un premier temps, les principes généraux liés à la collecte et à la gestion des données de production seront présentés. Ensuite, les différentes architectures de consolidation des données seront étudiées. Enfin, une comparaison pratique des approches fédérée et ETL sera réalisée au moyen de tests afin d'évaluer leurs performances respectives.

Chapitre 2

Contexte Technique

2.1 Les bases de données

Comme mentionné dans l'introduction, nous travaillerons avec 2 bases de données.

2.1.1 Base principale

La base de données principale est alimentée par la solution logicielle SPManager, qui utilise également cette dernière pour effectuer les différents affichages et traitements nécessaires.

Cette base centralise la majorité des données de production, incluant les mesures, la définition des mesures à réaliser, ainsi qu'un suivi partiel des pièces. Elle permet ensuite l'exploitation et l'analyse de ces informations.

La base est un serveur SQL Server hébergé au sein de l'entreprise et reçoit actuellement environ 50 000 lignes par jour.

L'accès à la base peut se faire soit directement via SQL pour certaines requêtes spécifiques, soit via le logiciel à travers de l'ORM Dapper, qui récupère les données pour l'exploitation et l'analyse.

L'alimentation de la base s'effectue majoritairement via SPManager : toutes nouvelles pièces, mesures ou configurations sont ajoutées par les modules correspondants de l'application, avec la possibilité de modifier ou de supprimer des informations. La suppression et les modifications ne sont pas définitives grâce à un système d'historique permettant de restaurer les données en cas de suppressions/changements involontaires ou d'erreur.

Table 2.1 – Résumé des caractéristiques de la base principale

Élément	**Description**
Type de base	SQL Server
Localisation	Serveur interne de l'entreprise
Volume	Environ 50 000 lignes par jour
Accès	SQL direct ou via SPManager (ORM Dapper)
Alimentation	SPManager (ajout, modification, suppression, le tout avec historique)

2.1.2 Base secondaire

La seconde base de données est directement reliée à la ligne de production automatisée. Elle récupère les informations générées par les équipements de la ligne afin de permettre leur exploitation en local.

Actuellement, cette base de données est isolée du reste du système d'information de l'entreprise. Elle est donc utilisée uniquement pour les besoins spécifiques liés au fonctionnement de la ligne automatisée.

Comme pour la base principale, le système de gestion de base de données utilisé est SQL Server. Toutefois, dans ce cas précis, la base est hébergée localement sur la machine associée à la ligne automatisée.

À l'heure actuelle, aucune information précise n'est disponible concernant le volume de données généré quotidiennement par cette base.

L'alimentation de la base de données se fait actuellement directement via des requêtes SQL exécutées par les systèmes associés à la ligne automatisée.

Table 2.2 – Résumé des caractéristiques de la base secondaire

Élément	**Description**
Type de base	SQL Server
Localisation	Machine locale associée à la ligne automatisée
Rôle	Stockage des données générées par la ligne automatique
Intégration au système	Base actuellement isolée du reste du système d'information
Volume	Non déterminé
Alimentation	Requêtes SQL exécutées par les systèmes de la ligne automatisée

Ces deux bases de données constituent la base du système étudié dans ce travail. La base principale centralise actuellement les données de production exploitées par le logiciel SPManager, tandis que la base secondaire contient les données générées par la nouvelle ligne automatisée mais reste isolée du reste du système.

Cette séparation soulève la question de la méthode la plus adaptée pour centraliser efficacement ces informations, ce qui constitue l'objet de l'étude présentée dans ce mémoire.

2.2 Architectures actuelles

2.2.1 Architecture globale

En ce qui concerne les architectures, il est possible de représenter l'ensemble du système par le schéma suivant :

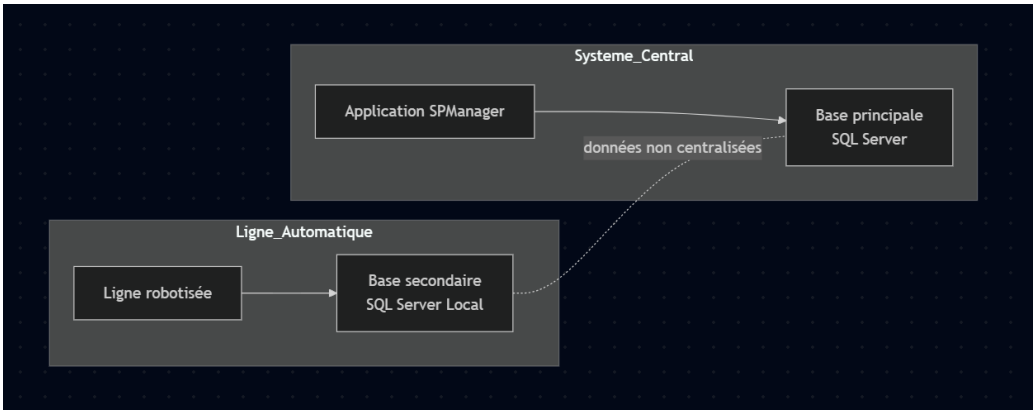

FIGURE 2.1 – Schéma de l'architecture globale du système d'information de la gestion de production

Ce schéma met en évidence que, pour le moment, les deux systèmes sont totalement séparés. L'application SPManager est reliée à la base de données principale afin de gérer l'ensemble des données liées à la production déjà en place.

La ligne automatisée, installée plus récemment, fonctionne quant à elle de manière locale et dispose de sa propre base de données, sans liaison avec le système principal.

2.2.2 Flux de données

L'architecture globale souligne la séparation du système en deux sous-systèmes distincts, chacun possédant son propre flux de données.

Flux SPManager

La figure 2.2 ci-dessous permet d'illustrer le flux de données de l'application SPManager :

FIGURE 2.2 – Schéma du flux de données du système SPManager

Le flux de données associé à SPManager est relativement simple. L'utilisateur saisit des informations ou effectue une demande d'accès à certaines données. L'application traite ensuite ces demandes et utilise l'ORM Dapper afin d'interagir avec la base de données.

Dans le cas d'une insertion ou d'une modification de données, la base de données exécute l'opération sans nécessairement renvoyer de résultat exploitable par l'utilisateur. En revanche, dans le cas d'une requête de consultation, la base de données renvoie les résultats correspondants.

Ces résultats sont alors récupérés par Dapper, qui effectue le mapping entre les données SQL et les objets C# utilisés dans l'application. Les données peuvent ensuite être exploitées par l'application et présentées à l'utilisateur.

Flux ligne automatisée

Tout comme précédemment, la figure 2.3 permet d'illustrer cette fois-ci le flux de données de la ligne automatisée :

FIGURE 2.3 – Schéma du flux de données de la ligne automatique

Le flux de données de la ligne automatique est encore plus simple, le contrôleur de ligne récupère toutes les informations nécessaires des capteurs et machines de la ligne, avant de les envoyer via logiciel ou script SQL dans la base secondaire, où les données pourront être exploitées de manière locale.

2.3 Stack technique

Enfin, en ce qui concerne la stack technique utilisée dans le système étudié, celle-ci se compose principalement de deux bases de données SQL utilisant SQL Server comme système de gestion de base de données.

Le système central repose également sur une application développée avec la technologie WPF (Windows Presentation Foundation), basée sur la plateforme .NET et utilisant le langage C#.

Plusieurs bibliothèques sont utilisées au sein de l'application afin de faciliter l'accès aux données et l'implémentation de certaines fonctionnalités. Parmi celles-ci, on retrouve notamment l'ORM léger Dapper, qui permet d'interagir efficacement avec la base de données SQL Server tout en conservant un accès direct aux requêtes SQL.

La stack technique utilisée peut être résumée comme suit :

- Système de gestion de base de données : SQL Server
- Framework applicatif : .NET
- Interface utilisateur : WPF
- Langage de développement : C#
- Accès aux données : ORM Dapper
- Gestion des dépendances : NuGet

Chapitre 3

État de l'art

La centralisation des données peut être réalisée à l'aide de différentes technologies et architectures. Elle soulève cependant des problématiques spécifiques, notamment en lien avec certains concepts clés de la gestion et de l'exploitation des données.

Ce chapitre a pour objectif de présenter les concepts et solutions existantes. Cela permettra de définir une base technique et théorique permettant de mieux comprendre et définir les tests qui seront effectués. Ce qui découlera sur une meilleure comparaison des deux architectures étudiées dans ce travail.

Dans ce chapitre, nous présenterons successivement :

— L'architecture fédérée
— Le processus ETL
— Les jointures distribuées
— La centralisation des données
— Les principaux problèmes de performances

3.1 L'architecture fédérée

Une architecture fédérée permet d'offrir une vue unique sur des données provenant de plusieurs systèmes différents, sans les centraliser physiquement. Chaque système reste autonome et expose une partie de ses données via des interfaces explicites, tandis qu'une couche logicielle ou un service d'intégration fournit un schéma global virtuel permettant aux utilisateurs et aux applications d'interroger l'ensemble comme s'il s'agissait d'une seule base.

Cette approche conserve l'autonomie des systèmes : chaque source garde son modèle, sa gestion des droits, son stockage ainsi que ses processus internes. La vue globale combine les différentes sources et est exposée à l'application, tout en évitant toute duplication de données.

L'architecture fédérée nécessite également une certaine transparence et interopérabilité entre les systèmes. Les requêtes doivent pouvoir être exécutées sans que l'utilisateur ou l'application ait à se soucier de l'emplacement physique des données, et les systèmes doivent pouvoir communiquer entre eux de manière fiable.

On distingue généralement deux formes standard d'architecture fédérée.

3.1.1 La fédération par vue virtuelle

Ce type de fédération utilise un médiateur chargé de construire une vue unifiée des différentes sources de données, sans stockage supplémentaire. Pour récupérer des informations, les requêtes sont envoyées au médiateur, qui les traduit et génère des sous-requêtes destinées aux différentes sources afin de produire le résultat final.

Ce fonctionnement permet d'éviter toute duplication de données et facilite une intégration rapide des systèmes tout en conservant leur autonomie. En revanche, les performances peuvent être limitées dans le cas de requêtes complexes, notamment lorsqu'elles impliquent des jointures entre plusieurs sources ou lorsqu'un grand volume de données doit être traité. Dans ce cas, les performances peuvent être fortement influencées par la latence réseau et les capacités des systèmes interrogés.

3.1.2 La fédération par hub central

Ce type de fédération repose sur un serveur central, souvent appelé hub. Celui-ci reçoit et coordonne les requêtes adressées aux différentes sources de données.

Afin d'améliorer les performances, le hub peut maintenir des pré-agrégats ou utiliser un cache partiel pour accélérer l'accès aux données. L'application interroge directement le hub, qui peut soit répondre à partir des

données déjà présentes dans le cache, soit interroger les sources externes si les informations demandées ne sont pas disponibles localement.

Cette approche permet d'améliorer les performances pour certaines requêtes fréquentes, mais elle introduit également une complexité supplémentaire dans la gestion du système et peut nécessiter un stockage intermédiaire des données.

3.2 ETL

Le processus ETL est une technique d'intégration de données couramment utilisée en informatique décisionnelle. Il permet d'extraire des données provenant de différentes sources, de les transformer afin de les adapter à un format commun, puis de les charger dans une base de données cible, généralement centralisée.

Ce processus se déroule en trois étapes principales : l'extraction, la transformation et le chargement des données.

3.2.1 Extract

L'étape d'extraction consiste à interroger les différentes sources de données afin de récupérer les informations nécessaires. L'ETL sélectionne les données pertinentes et les prépare pour les étapes suivantes du traitement.

3.2.2 Transform

Dans cette étape, on réutilise les données récupérées lors de l'extraction et on les transforme pour les adapter à la base cible. Cette transformation peut inclure plusieurs opérations.

- **Standardisation :**
  Lors de la standardisation, on définit les données à traiter, leur format et leur mode de stockage, cela permet de définir quels seront les étapes qui vont suivre.
- **Déduplication :**
  Pendant la déduplication, on va chercher les données redondantes et décider de leur exclusion et/ou suppression, permettant ainsi d'éviter les doublons sur des données déjà existantes.
- **Vérification :**
  On effectue des vérifications automatisées pour comparer les données similaires. Cela permet d'éliminer les données inexploitables et de signaler les anomalies des systèmes, applications ou données.
- **Tri :** Dans le cas d'un ETL visant à alimenter un DataLake, par exemple, on aura enfin une étape de tri pour séparer les données de BDD, de mail, etc. Cela définit leur prochaine destination et est aussi utilisé pour préparer les agrégations de données utilisées pour les rapports par exemple.

  Dans notre cas, étant donné que nous travaillerons sur le transfert d'une base de données vers une autre, le tri se limitera à définir quelles données vont dans quelles tables de la base cibles, et les agréger en conséquence.
- **Autres Tâches :** D'autres tâches peuvent être mises en place dans l'ETL pour répondre à des besoins spécifiques lors de l'ingestion des données. Ces dernières seront majoritairement des tâches à exécuter à chaque extraction qui doivent être automatisées et qui ne sont pas déjà gérées dans les étapes mentionnées ci-dessus.

3.2.3 Load

L'étape de chargement (Load) conclut le processus ETL en insérant les données extraites et transformées dans la base de données cible. Cette étape peut être réalisée de différentes manières selon les besoins du système : insertion de nouvelles données, mise à jour de données existantes ou remplacement complet de certaines tables.

Le chargement peut également inclure des contrôles supplémentaires afin de garantir l'intégrité des données dans la base cible. Une fois cette étape terminée, les données deviennent accessibles pour les applications et autres systèmes les nécessitant.

3.3 Jointures distribuées

Dans les systèmes où les données sont réparties sur plusieurs bases ou plusieurs serveurs, les opérations de requête peuvent nécessiter de combiner des informations provenant de différentes sources. Dans ce cas, on parle de jointures distribuées. Une jointure est une opération fondamentale dans les bases de données relationnelles qui consiste à associer des données provenant de plusieurs tables selon un lien logique de données qui est défini par une condition appelée prédicat. Lorsque les tables impliquées sont stockées sur des systèmes différents, la jointure devient distribuée.

Les jointures distribuées apparaissent notamment dans les architectures de bases de données distribuées qui sont généralement présentes dans les systèmes fédérés. Une base de données distribuée stocke ses données non pas sur un seul, mais sur plusieurs nœuds du système, il y a donc une séparation physique des emplacements de stockage utilisés par la base de données. Cette architecture est très utile dans les cas où une base de données particulière doit être accessible par des utilisateurs au niveau mondial par exemple. Elle nécessite une gestion telle que pour les utilisateurs, elle fonctionne comme une base unique.

La réalisation d'une jointure distribuée implique généralement plusieurs étapes. Tout d'abord, les données nécessaires doivent être localisées dans leurs sources respectives, qui correspondent chacune à un nœud du système. Ensuite, certaines parties des tables peuvent être transférées vers un site commun afin d'effectuer la jointure localement. Enfin, les résultats obtenus sont renvoyés à l'application ou au système qui a effectué la requête. Ce processus peut engendrer un coût important en termes de communication réseau et de traitement.

Un des principaux défis des jointures distribuées concerne le transfert de données entre les différents systèmes. Lorsque les tables à joindre sont volumineuses, le transfert de grandes quantités de données peut dégrader fortement les performances, notamment si le réseau n'est pas assez puissant pour supporter une telle quantité de mouvement de données en simultané. Dans certains cas, les systèmes doivent regrouper une partie des données sur un même nœud afin d'effectuer la jointure localement, ce qui peut entraîner une surcharge du réseau.

Pour limiter ces coûts, plusieurs stratégies d'optimisation peuvent être utilisées. Par exemple, il est possible de filtrer les données directement sur chaque source avant leur transfert afin de réduire le volume des informations à transmettre. D'autres approches consistent à exécuter certaines opérations en parallèle sur plusieurs nœuds du système. Ces techniques sont notamment utilisées dans les systèmes de traitement de données distribués à grande échelle.

Dans le cadre de ce travail, ces principes sont étudiés dans un contexte simplifié, à l'aide de bases de données simulées qui reste proche du contexte présent dans l'entreprise. L'objectif n'est pas de reproduire l'ensemble de la complexité des systèmes distribués industriels de plus grandes tailles, mais d'observer le fonctionnement et les implications des jointures entre plusieurs sources de données distinctes dans notre contexte spécifique.

3.4 Centralisation des données

La centralisation des données consiste à regrouper des informations provenant de différentes sources dans un référentiel unique et unifié. Cette approche limite la dispersion des données sur plusieurs systèmes ou supports. On privilégie un point d'accès central qui devient la "source unique de vérité" pour tous. Cela garantit que les utilisateurs accèdent à un ensemble de données cohérent et partagé.

Une approche couramment utilisée pour mettre en place cette centralisation repose sur l'utilisation d'un entrepôt de données (Data Warehouse). Un Data Warehouse permet de stocker des données provenant de multiples sources dans un système unique, afin de faciliter leur analyse et leur exploitation.

Les sources de données peuvent être diverses, telles que des systèmes ERP, des CRM ou encore différentes bases de données. Les informations sont généralement extraites depuis ces systèmes, transformées afin d'être mises dans un format cohérent, puis chargées dans la base centrale constituant l'entrepôt de données.

3.4.1 Avantages

La centralisation des données présente plusieurs avantages.

Tout d'abord, elle améliore la prise de décision. Le regroupement des données dans un référentiel unique facilite l'accès à des informations pertinentes et cohérentes, permettant aux entreprises de prendre des décisions plus éclairées et de réagir plus rapidement face aux changements ou aux problèmes potentiels.

La centralisation permet également de standardiser les formats de données et de réduire les doublons, ce qui contribue à améliorer la qualité globale des informations disponibles.

Elle favorise aussi la collaboration entre les équipes. L'accès à une source commune d'information permet aux différents acteurs d'une organisation de travailler à partir des mêmes données, ce qui facilite les projets impliquant plusieurs départements.

Enfin, l'agrégation de données provenant de différentes sources (ERP, CRM, bases de données, etc.) permet d'obtenir une vision plus globale de l'activité de l'entreprise.

3.4.2 Inconvénients

Cependant, ces avantages s'accompagnent également de certaines contraintes.

La mise en place d'une solution de centralisation telle qu'un Data Warehouse nécessite souvent une infrastructure importante ainsi que des compétences techniques spécialisées pour assurer son déploiement et sa maintenance.

Les coûts initiaux peuvent également être élevés, notamment en raison des investissements nécessaires dans les infrastructures, les outils logiciels ou encore les prestations liées à la mise en place du système.

Enfin, la centralisation peut parfois devenir un point de congestion si le volume de données à traiter devient trop important, ce qui peut entraîner une dégradation des performances du système.

3.5 Problèmes de performances

Les approches présentées précédemment, qu'il s'agisse de la fédération, de l'utilisation d'ETL, de jointures distribuées ou de centralisation, permettent toutes d'intégrer et d'exploiter des données provenant de sources multiples. Cependant, elles introduisent également des défis importants en termes de performances, liés à la latence, au volume de données transféré, à l'optimisation des requêtes ou encore à la répartition inégale des données. Cette section présente ces principaux problèmes, qui constituent un enjeu central dans le choix de la solution à mettre en œuvre.

3.5.1 Coût des communications réseau

Dans une architecture fédérée, les requêtes doivent souvent passer par plusieurs sous-systèmes situés sur des serveurs différents. Chaque appel réseau ajoute de la latence et peut devenir un goulot d'étranglement si une source est lente ou fortement sollicitée.

Ainsi, la performance globale dépend non seulement de la vitesse de chaque sous-système, mais aussi de la qualité et de la charge du réseau qui les relie. Les architectures de données distribuées, comme la fédération, doivent transférer des données entre plusieurs sources pour répondre à une requête, ce qui peut avoir un impact significatif sur la performance globale, notamment en termes de latence réseau et de charge de communication.

3.5.2 Coût des jointures distribuées

Une jointure est une des opérations les plus coûteuses dans une base de données. Quand elle doit être effectuée sur des sources réparties, cela impose de déplacer des données ou de calculer des agrégats intermédiaires avant de produire le résultat.

Contrairement aux jointures locales, les jointures distribuées ne bénéficient pas pleinement des mécanismes d'optimisation présents dans les systèmes de gestion de bases de données centralisés, tels que les index, les plans d'exécution optimisés ou les statistiques locales. Ce manque d'optimisation combiné à la nécessité de coordination entre plusieurs sources peut entraîner des coûts significatifs en termes de temps de calcul et de bande passante réseau.

3.5.3 Difficulté d'optimisation des requêtes

Dans un environnement centralisé, le moteur de base de données peut exploiter pleinement des mécanismes d'optimisation tels que les index, les statistiques locales et les plans d'exécution optimisés. En revanche, dans une architecture fédérée ou distribuée, l'optimisation devient beaucoup plus complexe. Il faut alors prendre en compte la répartition des données entre plusieurs sources, la latence réseau, ainsi que le coût des transferts de données pour produire un plan d'exécution efficace

Cette complexité implique souvent des ressources supplémentaires pour mettre en place des solutions capables de compenser le manque d'optimisation automatique et garantir des performances acceptables dans le traitement des requêtes distribuées.

3.5.4 Déséquilibre des données

Dans les systèmes distribués, certaines partitions de données peuvent être beaucoup plus volumineuses ou sollicitées que d'autres, entraînant un déséquilibre de la charge de travail entre les différents nœuds. Ce phénomène, souvent appelé data skew, peut provoquer une surcharge sur certains serveurs tandis que d'autres restent sous-utilisés, ce qui dégrade les performances globales et augmente le temps de réponse des requêtes.

La gestion du data skew est un enjeu crucial dans les architectures fédérées ou distribuées, car elle impacte directement l'efficacité du traitement des requêtes et la répartition optimale des ressources.

Chapitre 4

Architecture des solutions

Dans ce travail, nous allons étudier deux des architectures présentées précédemment :

L'extraction des données de la base secondaire vers la base centralisée via un processus ETL, ainsi que la mise en place d'un système de fédération permettant d'interroger les bases de données séparément et de produire un résultat au niveau d'un médiateur.

Nous allons, dans un premier temps présenter les architectures qui seront mises en place dans notre cas précis, afin de mieux comprendre le contexte technique utilisé pour les tests.

4.0.1 Architecture ETL

Le cas étudié au sein de l'entreprise étant relativement simple, l'architecture mise en place reste également peu complexe, comme le montre la figure 4.1 ci-dessous.

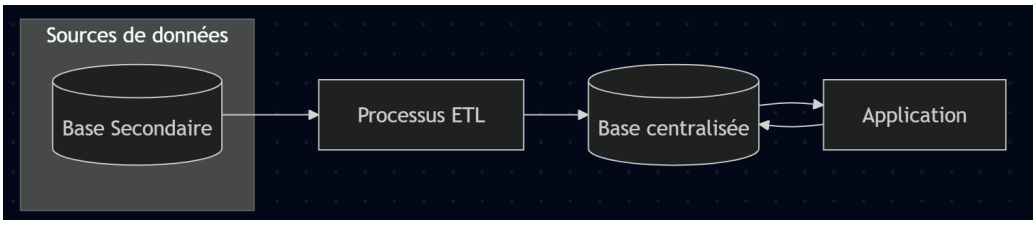

Figure 4.1 – Schéma de l'architecture ETL

Techniquement, cette architecture se constitue de deux bases de données relationnelles fonctionnant avec le SGBDR SQL Server ; l'ETL sera mis en place grâce à une solution C#. L'application est-elle aussi en C#, comme mentionné dans le chapitre du contexte technique.

Le fonctionnement de l'architecture est le suivant. Le processus ETL récupère les données provenant de la base secondaire située au niveau de la ligne automatique. Ces données sont ensuite transformées afin d'être adaptées au modèle de la base centralisée, puis chargées dans cette dernière.

L'application C# peut alors exploiter ces données centralisées, en les combinant avec les données de production déjà présentes dans la base principale.

4.0.2 Architecture Fédérée

À l'instar de l'architecture ETL, l'architecture fédérée reste peu complexe, comme l'illustre la figure 4.2 ci-dessous :

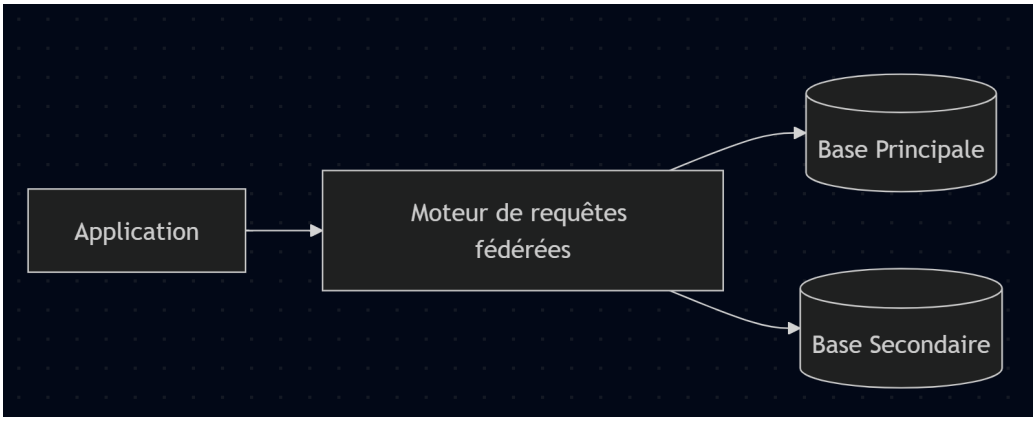

FIGURE 4.2 – Schéma de l'architecture fédérée

Cette architecture utilisera elle aussi deux bases de données relationnelles fonctionnant sur le SGBDR SQL Server, le médiateur serait là encore mis en place grâce à une solution en C#.

Dans cette architecture, le moteur de requêtes fédérées reçoit la requête provenant de l'application. Il traduit ensuite cette requête en plusieurs sous-requêtes permettant d'interroger séparément les deux bases de données sur les informations nécessaires.

Une fois les résultats récupérés, le moteur fédéré combine les données obtenues afin de produire un résultat unique qui est ensuite renvoyé à l'application.

Ces deux approches sont très différentes dans leurs avantages et contraintes, que l'on a abordés plus haut.

4.0.3 Synthèse de chapitre

Les deux approches étudiées présentent des avantages et des contraintes différentes.

L'architecture basée sur un processus ETL se concentre sur la centralisation des données, ce qui permet d'exécuter les requêtes au sein d'une base unique et de bénéficier pleinement des mécanismes d'optimisation proposés par les systèmes de gestion de bases de données.

À l'inverse, l'architecture fédérée permet d'éviter les problématiques liées à la standardisation et à la transformation des données entre les différentes bases. Cependant, cette approche introduit des difficultés supplémentaires en matière d'optimisation des requêtes, qui doivent être prises en charge par le médiateur afin de garantir les performances recherchées.

C'est donc grâce aux tests que nous nous apprêtons à réaliser que nous pourrons déterminer quelle solution est la plus adaptée dans notre cas.

Chapitre 5

Méthodologie de test

5.1 Introduction

L'objectif de cette section est de définir la méthodologie qui sera utilisée afin d'évaluer et de comparer objectivement les performances de deux architectures d'intégration de données. D'une part, une architecture ETL reposant sur une base de données centralisée, et d'autre part, une architecture de requêtes fédérées permettant d'interroger plusieurs bases de données sans mouvement préalable de ces dernières.

Ces deux approches répondent à des contraintes bien différentes. L'architecture ETL consolide les données dans une base centralisée avant leur exploitation, ce qui implique un coût d'ingestion mais garantit des temps de lecture optimisés par la suite. L'architecture fédérée, à l'inverse, interroge les sources en temps réel, supprimant le besoin de synchronisation mais introduisant une latence distribuée lors de l'exécution des requêtes.

Afin d'isoler la variable « architecture » et de produire des résultats comparables et reproductibles, l'ensemble des tests sera réalisé dans un environnement technologique homogène, correspondant à l'infrastructure existante du projet : Microsoft SQL Server pour les bases de données, et C# (.NET) pour les applications et les solutions d'intégration.

5.2 Environnement de test

5.2.1 Infrastructure matérielle et logicielle

Pour garantir l'équité des mesures, les deux architectures sont déployées sur une même machine physique, avec la même configuration pour chaque série de tests. Aucune autre application ne sera lancée pendant les tests, dans le but d'éliminer les interférences de ces dernières.

Table 5.1 – Configuration de l'environnement de test

Composant	**Configuration**
Système d'exploitation	Windows 11 Home 24H2 (Build 26200)
CPU	Intel Core i9-14900HX — 24 cœurs/32 threads — 2,20 GHz (boost max 5,80 GHz)
Mémoire RAM	64 Go DDR4-5600 (2 × 32 Go, Dual Channel, XMP)
Stockage	SSD NVMe
SGBD	Microsoft SQL Server 2025 Developer Edition (RTM) 17.0.1000.7
Framework applicatif	.NET 9 / C# 13
Outil de mesure serveur	SQL Server Extended Events, sys.dm_exec_query_stats
Outil de mesure client	System.Diagnostics.Stopwatch(.NET)

5.2.2 Implémentation de l'architecture ETL

Dans le cadre de ces tests, l'architecture ETL est implémentée sous la forme d'une pipeline C# personnalisée. Ce choix permet de respecter les contraintes qui seraient rencontrées dans le cas de l'implémentation de cette solution dans l'entreprise et permet d'éviter l'introduction d'un outil ETL tiers.

La pipeline repose sur deux composants principaux. Le premier est la classe SqlBulkCopy qui vient du namespace System.Data.SqlClient, elle permet des insertions en masse vers SQL Server avec des performances bien supérieures aux insertions ligne par ligne. Le second est un Windows Service .NET permettant de coordonner et de séquencer l'exécution périodique du pipeline via la bibliothèque Quartz.NET.

Le processus ETL suit les trois phases classiques :

— Lecture des données depuis la base secondaire via Microsoft.Data.SqlClient avec un DataReader en mode streaming pour limiter la consommation mémoire ;

— – Transformation : application des règles de gestion en mémoire (normalisation, jointures légères, calculs dérivés) ;

— – Chargement : écriture dans la base centralisée via SqlBulkCopy avec validation transactionnelle.

5.2.3 Implémentation de l'architecture fédérée

L'architecture de requêtes fédérées sera évaluée grâce à deux implémentations différentes, les deux étant réalisables dans l'écosystème Microsoft sans avoir recours à des dépendances externes :

Option A - Linked Servers SQLServer : SQLServer expose nativement un mécanisme de serveurs liés (Linked Servers), ces derniers permettent d'exécuter des requêtes T-SQL cross-instances. Le moteur de requêtes fédérée devient alors SQL Server lui-même, et l'application C# ne verra qu'une connexion unique. C'est la manière la plus simple d'implémenter l'architecture fédérée tout en répondant aux contraintes liées à Windows.

Option B - Médiation C# en mémoire : La deuxième implémentation repose sur un service C# qui recevra les requêtes de l'application. Ce dernier les décompose en sous-requêtes qu'il exécute en parallèle sur chaque base via Microsoft.Data.SqlClient. Puis, grâce au module LINQ, il assemble les résultats en mémoire. Cette approche démontre plus fidèlement l'utilisation d'un moteur de médiation custom et se trouve être un cas d'étude intéressant pour mesurer la différence de coût entre une jointure applicative et une jointure côté serveur.

Ainsi, l'évaluation des deux options permet, au-delà de la comparaison ETL vs fédération, de montrer aussi l'impact du lieu d'exécution de la jointure (moteur SQL vs mémoire applicative).

5.3 Jeux de données

5.3.1 Choix du benchmark

Les jeux de données utilisés sont générés de manière semi-aléatoire dans un projet maison DataGenerator via une seed fixe pour garantir la reproductibilité des tests entre les exécutions.

Le schéma retenu pour les tests est constitué d'un sous-ensemble des tables présentes dans la base principale. On ne garde pour ce dernier que les tables impliquées dans les requêtes les plus critiques que nous aborderons en section 5.4.

Les tables sans lien avec ces requêtes (*Provider*, *Assemblage*, *LienPersonnesPostes*, *EtiquettesLog*) ont été exclues du périmètre sans impact sur la représentativité des mesures.

Le générateur de données DataGenerator produit des données pour remplir le schéma retenu, ce dernier effectue cette opération à différentes échelles. Cela permet de comparer le comportement des requêtes sur des jeux de données de plus en plus volumineux, et donc d'observer comment elles réagissent face à la montée en charge.

Pour ces tests, nous utiliserons trois facteurs d'échelle pour observer le comportement des architectures en fonction du volume :

Niveau	**Facteur d'échelle**	**Volume approx.**	**Lignes (Mesures)**
Petit	Small	∼10 Mo	∼20 000
Moyen	Medium	∼100 Mo	∼250 000
Grand	Large	∼1 Go	∼3 000 000

Table 5.2 – Facteurs d'échelle retenus pour les tests

5.3.2 Distribution des données entre les bases

Pour que les tests soient représentatifs, la base secondaire actuellement présente dans les architectures décrites dans les chapitres précédents n'a pas été retenue. Sa structure étant trop éloignée de la base principale et son intégration en production pas encore réalisée, il a été jugé plus pertinent de simuler la séparation inter-base à partir du schéma retenu mentionné en 5.3.1.

Ce schéma est scindé en trois bases de la façon suivante :

- **Base A** — `Bench_Referentiel` : tables de référentiel à faible volumétrie et forte stabilité — *Type_ Piece, Type_ mesure, Lien_ type_ mesure_ type_ piece, Postes, Personnes, Nomenclature, Priorites*
- **Base B** — `Bench_Production` : tables opérationnelles à forte volumétrie et insertions fréquentes — *Piece, Infos, Mesures, Historique, Derogations, Etiquettes*
- **Base C** — `Bench_Centralized` : base de destination de l'architecture ETL, contenant le schéma complet des deux bases réunies. Elle constitue une simplification volontaire de la base de destination réelle, sans impact sur la validité des mesures comparatives.

5.4 Scénarios de requêtes

Sept scénarios de tests sont définis, ils permettent de couvrir un éventail représentatif des charges que l'on peut rencontrer en production. Chaque scénario est exécuté sur les trois niveaux de volumes définis précédemment.

Test	**Type de requête**	**Description**	**Architectures concernées**
T1	SELECT simple	SELECT sur la table Piece avec filtre Is_Active et ORDER BY ID_SN	ETL, Fédéré A+B
T2	SELECT multi-jointures	Récupération des mesures par type de pièce, 4 jointures inter-tables/bases	ETL, Fédéré A+B
T3	SELECT analytique (CTE + fenêtrage)	Historique complet des mesures d'une pièce via CTE + LAG	ETL, Fédéré A+B
T4	INSERT unitaire	Insertion d'une mesure avec récupération de l'ID généré	ETL, Fédéré A+B
T5	Charge en rafale (lecture)	Burst de requêtes T2 en parallèle, montée de 10 à 200 workers	ETL, Fédéré A+B
T6	Ingestion batch	Transfert batch de Bench_Production vers Bench_Centralized via SqlBulkCopy	ETL uniquement
T7	Charge en rafale (écriture)	Burst de requêtes T4 en parallèle, montée de 10 à 200 workers	ETL, Fédéré A+B

Table 5.3 – Scénarios de test du benchmark

Le test T6 est spécifique à l'architecture ETL, il n'a pas d'équivalent direct dans l'architecture fédérée, cette dernière n'ayant pas la nécessité d'avoir une pipeline d'ingestion. Cependant, inclure ce test est essentiel pour évaluer le coût opérationnel réel de l'ETL, qui doit être intégré dans l'analyse globale des deux architectures.

5.5 Métriques mesurées

5.5.1 Métriques de performance

Les métriques suivantes seront collectées pour chaque exécution des tests :

- Temps de réponse end-to-end (ms) : mesuré côté application via System.Diagnostics.Stopwatch, du déclenchement de la requête à la réception du dernier enregistrement. Cette métrique représente ce que perçoit réellement l'utilisateur final.
- Temps d'exécution côté serveur (ms) : capturé via les vues de gestion dynamique SQL Server (sys.dm_exec_query_stats ou les Extended Events, pour isoler le traitement moteur du temps réseau et applicatif.
- Débit d'ingestion (lignes/seconde) : applicable uniquement au test T6, mesuré nativement via la propriété RowsCopied de SqlBulkCopy.
- Utilisation CPU (%) : moyenne pendant la durée du test, collectée via les compteurs de performance Windows (PerformanceCounter .NET).
- Consommation mémoire (Mo) : pic mémoire du processus applicatif et du service SQL Server pendant l'exécution.
- Lectures logiques et physiques : extraites depuis sys.dm_exec_query_stats, ces métriques permettent d'évaluer l'efficacité des plans d'exécution indépendamment des performances I/O.

Test	Volume	Type de requête	Métrique principale	Architectures
T1	Small/Medium/Large	SELECT simple	Latence end-to-end	ETL / Fédéré A+B
T2	Small/Medium/Large	SELECT multi-jointures	Latence + CPU	ETL / Fédéré A+B
T3	Small/Medium/Large	SELECT analytique complexe	Latence + Lectures logiques	ETL / Fédéré A+B
T4	Small/Medium/Large	INSERT unitaire	Latence + Mémoire	ETL / Fédéré A+B
T5	Small/Medium/Large	Charge en rafale	Débit (req/sec)	ETL / Fédéré A+B
T6	Small/Medium/Large	Ingestion batch	Débit (lignes/sec)	ETL uniquement

Table 5.4 – Récapitulatif des métriques par scénario

5.5.2 Protocole de mesure et traitement statistique

Afin d'obtenir des résultats statistiquement fiables et de limiter l'impact des variations environnementales (tel que le cache SQL ou la charge OS), le protocole suivant sera appliqué

- Une exécution à blanc (warm-up) précède chaque série de mesures, afin d'initialiser les caches SQL Server et les plans d'exécution ;
- Chaque scénario est exécuté dix fois consécutivement dans les mêmes conditions ;
- La médiane des dix mesures est retenue comme valeur de référence, plutôt que la moyenne arithmétique, afin de limiter l'influence des valeurs aberrantes (pics de charge, GC .NET) ;
- L'écart interquartile (P25–P75) est calculé pour déterminer la stabilité des mesures ;
- Le cache SQL Server est vidé manuellement (DBCC DROPCLEANBUFFERS) avant chaque nouvelle série pour garantir des conditions initiales identiques sur les tests de volume croissant.

5.6 Limites et biais de la méthodologie

Toutes méthodologie de test possède des limites qu'il est important d'identifier afin de mieux interpréter les conclusions qui en seront tirées.

Premièrement, les résultats peuvent être influencés par la configuration matérielle retenue. Des proportions de CPU/RAM différentes, ou un stockage HDD au lieu d'un SSD NVMe, pourraient modifier significativement les résultats relatifs entre les deux architectures, notamment pour les tests présentant de fort volume d'I/O.

Deuxièmement, l'implémentation C# du pipeline ETL et du médiateur fédéré est une implémentation de référence non optimisée. Des gains de performances conséquents seraient envisageables grâce à un parallélisme accru, le pooling de connexions ou encore la compression de flux réseau. Sans pour autant remettre en cause la comparaison architecturale.

Enfin, le choix d'utiliser une base centralisée réunissant directement le schéma complet des deux bases constitue une simplification volontaire par rapport à l'architecture ETL réelle. Dans un contexte de production, la base de destination ne contiendrait initialement que les tables de la base principale, les tables du référentiel y étant progressivement ingérées par le pipeline. Cette simplification influence très légèrement les mesures de T6 — le débit d'ingestion mesuré porte sur la table Mesures uniquement, alors qu'un pipeline complet inclurait également les tables de référentiel. Les comparaisons entre architectures restent cependant pertinentes et exploitables, cette limite n'affectant pas les scénarios T1 à T5 qui constituent le cœur de l'analyse entre les architectures.

Chapitre 6

Résultats des tests et Analyse :

Maintenant que le plan d'exécution pour les tests a été détaillé, nous pouvons exploiter les résultats obtenus pour déterminer quelle architecture est la plus adaptée en nous penchant sur les différents scénarios et leur résultat.

6.1 Condition d'exécution

Avant de passer aux différentes analyses, nous allons voir les conditions dans lesquelles les tests ont été effectués.

Dans un premier temps, tous les tests ont été faits sur une machine locale dont les spécifications sont déjà présentées en section 5.2.1. Cela implique que certaines mesures peuvent être affectées par des variables extérieures telles que des processus Windows qui pourraient augmenter la consommation CPU mesurée.

Le réseau utilisé quant à lui, est un réseau virtuel. On passe en effet par une interface de loopback. Pour simplifier, Windows simule une connexion réseau complète entre deux processus présents sur la même machine, ici notre ordinateur utilisé pour les tests. C'est notamment la fédération via LinkedServer qui exploite ledit réseau, cela la rend d'autant plus sensible aux contention CPU (attente du CPU quand le nombre de threads actifs dépasse le nombre de cœurs disponibles) ainsi qu'au scheduler Windows (c'est ce dernier qui décide quel thread accède à un cœur du CPU et quand, ce qui détermine quels processus peuvent avancer et lesquels doivent attendre). L'ETL et la fédération par médiation C# sont moins sensibles aux bruits de mesure causés par ces paramètres car ces dernier n'exploitent le CPU que pour exécuter les requêtes vers le client, ainsi qu'effectuer les jointures côté logiciel dans le cas de la médiation C#.

On applique le même procédé de mesure que présenté en 5.5.2, c'est-à-dire une exécution à blanc avant les mesures, suivi de dix exécutions consécutives du scénario, en vidant manuellement le cache SQL Server entre chaque exécution pour garantir des conditions initiales identiques entre chaque run. On retient la médiane de chaque série ainsi que l'IQR dans le cas du temps de réponse end-to-end, ce qui garantit des résultats robustes malgré les potentiels bruits de mesures.

Enfin, les tests étant effectués sur 3 volumes permettent de prendre conscience des différences architecturales et d'avoir une idée solide de leur tendance, et ce indépendamment des valeurs absolues obtenues, ces dernières restant propres à l'environnement de test.

6.2 T1 - Lecture simple

Maintenant que le contexte réel des tests est bien en place, nous pouvons analyser les différents résultats obtenus. Étant donné que chaque requête a pour but de mesurer des variables différentes, elles seront abordées par type de requêtes. Les différentes conclusions seront ensuite synthétisées, ce qui permettra de déterminer les meilleurs cas d'usage pour chaque architecture.

La requête T1 est un SELECT simple qui vise à sélectionner toutes les pièces de la table "Piece". Étant donné que la requête est simple (elle ne nécessite pas de jointures inter-base ou de transfert de données entre serveurs), elle a donc pour but d'établir des premières mesures qui constitueront un socle de référence pour les analyses à venir.

Pour cette requête, on observe le temps d'exécution de l'envoi de la requête, jusqu'à la réception du dernier enregistrement, cela permet d'avoir une première idée des tendances que l'on devrait apercevoir plus tard dans les analyses. La figure 6.1 illustre ces mesures :

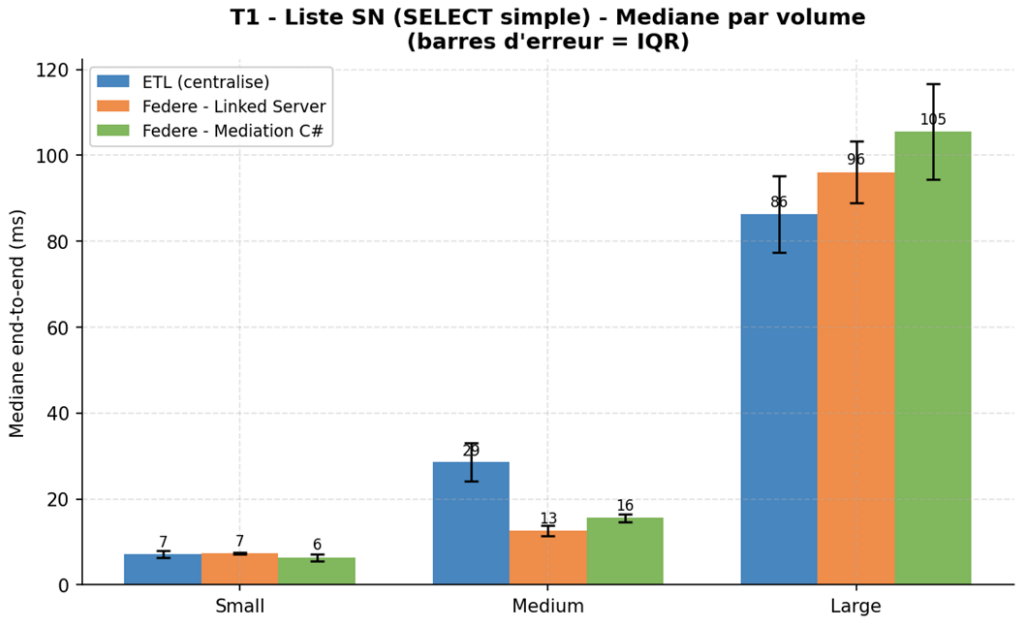

FIGURE 6.1 – Graphique du temps d'exécution end-to-end de la requête T1

On observe ici des écarts visibles, mais faibles, l'écart le plus notable étant l'ETL pour le volume medium. On peut supposer que ces différents écarts sont dus aux bruits de mesures mentionnés précédemment. La figure 6.2 appuie ces suppositions :

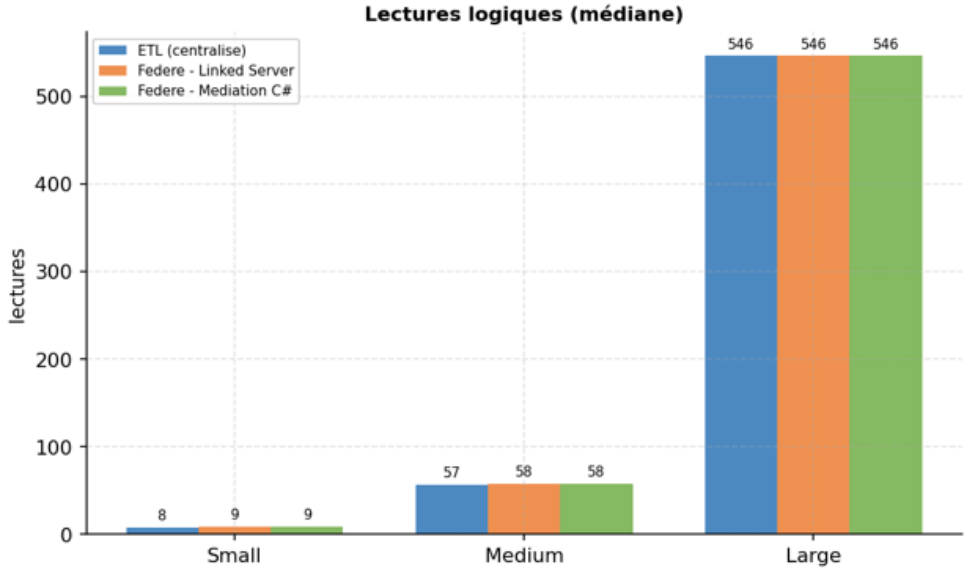

FIGURE 6.2 – Graphique des lectures logiques de la requête T1

On remarque que la charge de travail côté SQL est la même à une lecture près pour les 3 architectures. Cela suggère que les écarts observés relèvent davantage du bruit de mesure que des différences architecturales.

On en déduit que pour un SELECT simple, les trois architectures se valent, constituant ainsi un bon socle pour appuyer l'analyse des prochains types de requêtes. Il est donc maintenant plus simple d'étudier les cas plus complexes susceptibles de mettre en valeur des différences architecturales.

6.3 T2 & T3 - Requêtes complexes

La requête T2 est la première requête complexe présente dans les tests, elle récupère les mesures par type de pièces, ce qui dans notre architecture nécessite 4 jointures inter-tables dans le cas de l'ETL, inter-base dans le cas de la fédération. Cette requête devrait mettre en valeur une différence de tendance entre les architectures au niveau de l'utilisation CPU notamment.

L'ETL verra les jointures effectuées en local par SQL Server, du côté fédéré, le LinkedServer verra des jointures inter-bases, et la médiation C# fait la jointure du côté applicatif à partir de deux requêtes distinctes à destination de chacune des bases. Cela implique que l'ETL sera l'architecture exploitant le mieux les optimisations SQL Server sur les jointures, LinkedServer en profitera partiellement, la nature inter-base limitant l'accès qu'a SQL Server aux statistiques de la base distante, ce qui réduit l'efficacité des plans d'exécution. Enfin la médiation C# déplace donc la charge CPU côté application, au prix d'un coût mémoire supplémentaire et de l'inaccessibilité aux optimisations SQLServer.

La requête T3 est la deuxième requête complexe des tests. Elle récupère l'historique complet des mesures d'une pièce, elle combine les techniques de CTE, fenêtrage et LAG. Elle est particulièrement révélatrice pour la métrique des lectures logiques, notamment grâce aux notions de fenêtrage et LAG. Cela force SQLServer à garder toutes les lignes chargées pour les ordonner et pouvoir les parcourir de manière séquentielle, notamment en raison du LAG qui force la comparaison entre la ligne prise en compte par le fenêtrage et la ligne précédente dans notre cas.

Pour ETL, cela aura un impact négligeable, encore une fois dû à la colocalisation des données, les optimisations SQL Server limiteront significativement le nombre de lectures nécessaires. La fédération cependant doit rapatrier les données avant d'effectuer le traitement, les lectures logiques augmentent donc grandement bien que leur mécanisme de rapatriement soit différent, LinkedServer passant par le loopback TCP tandis que la médiation C# les récupère grâce à deux requêtes distinctes.

Analysons maintenant les résultats obtenus pour chaque requête

6.3.1 T2 - Jointure complexe

Comme présenté précédemment, la requête T2 se focalise sur l'analyse des performances CPU, on s'appuie ici sur le graphique 6.3 comme cas d'étude :

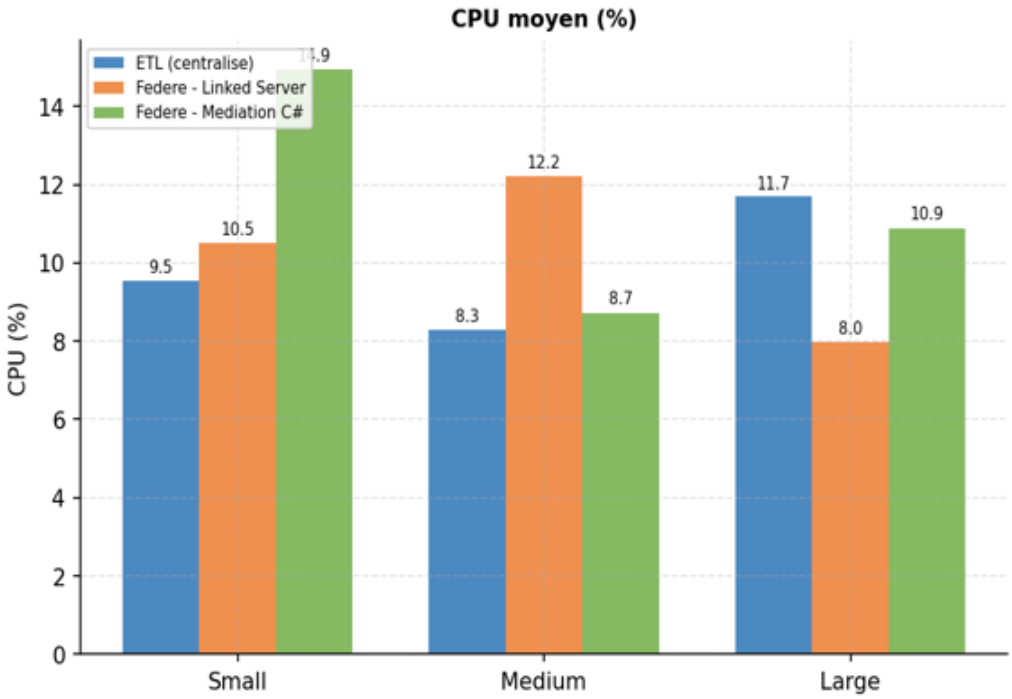

FIGURE 6.3 – Graphique de la charge CPU pour T2

On observe que les différences en termes d'utilisation CPU sont très légères, la variation la plus grande étant de 5%. Encore une fois on suppose que cela proviendrait majoritairement des bruits de mesures et non pas d'une différence architecturale notable.

La mesure CPU étant faite sur la globalité de l'utilisation de ce dernier, on ne peut isoler la charge propre à chaque architecture, cela constitue une limite de la méthodologie qui empêche de déterminer quelle hypothèse est juste.

Il serait pertinent de reproduire le test sur une machine dédiée bien moins sujette aux bruits de mesures pour valider ou invalider cette hypothèse, puis mettre en place des mécanismes de vérification de la charge architecturale sur le CPU pour vérifier l'hypothèse de la limite méthodologique. Cela montrerait définitivement si T2 est réellement pertinent ou non pour la mesure de la charge CPU.

Le graphique sur les lectures logiques que l'on récupère de ce test est cependant particulièrement pertinent, il sera étudié grâce à la figure 6.4 ci-dessous :

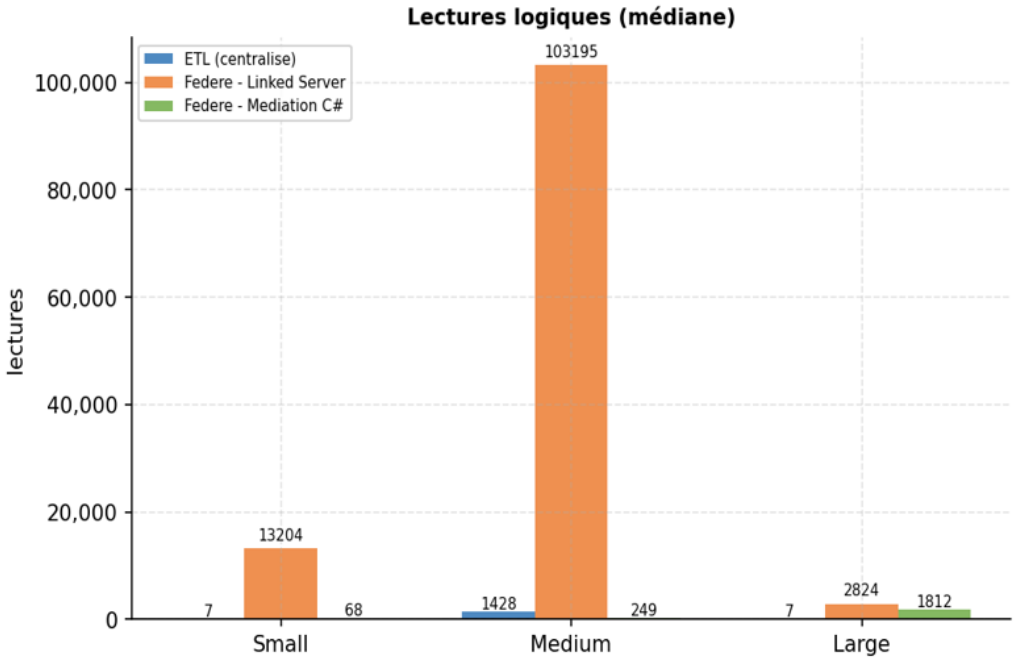

FIGURE 6.4 – Graphique des lectures logiques pour T2

On remarque ici une tendance bien marquée : l'ETL possède des valeurs très faibles, LinkedServer possède les valeurs les plus élevées, enfin la médiation C# suit une évolution proportionnelle au volume. Ce comportement est observable sur les trois volumes.

Le pic de lectures logiques sur le volume medium pourrait s'expliquer si on explore les possibles facteurs venant de SQL Server. On remarque que c'est le seul test où ETL monte en nombre de lectures et dépasse la fédération par médiation C#. La médiation, elle, suit sa tendance de croissance linéaire. Étant la seule qui n'est pas affectée par les plans d'exécution utilisés par SQL Server pour optimiser les jointures, cela renforce l'hypothèse que la raison se trouve côté SQL Server.

Comme mentionné, SQLServer utilise des plans d'exécution pour optimiser les jointures de la requête T2, qu'elle soit locale comme pour l'ETL ou distribuée dans le cas du LinkedServer. Il est donc possible que dans cette tranche de volume précisément ($\tilde{2}$50 000 mesures), SQL Server ai fait le choix d'un plan d'exécution moins optimisé que prévu pour traiter la requête. ETL et LinkedServer utilisant ces mécanismes d'optimisation pleinement ou partiellement, ils s'en trouvent affectés et cela se reflète sur leur nombre de lectures logiques anormales.

Cet impact est d'autant plus ressenti sur les mesures du LinkedServer car ce dernier possède un accès limité aux statistiques de la base distante, ce qui accroît les chances que SQLServer choisisse un plan d'exécution moins optimisé.

La médiation, quant à elle, effectuant les jointures côté applicatif, cette dernière n'est pas impactée.

Le fait que les tendances soient de nouveau présentes lors du test de large appuie d'autant plus l'hypothèse que l'erreur se trouve côté SQL Server.

Il serait possible de tester cette hypothèse de manière plus rigoureuse en explorant des cardinalités avoisinant celles utilisées pour le volume medium, permettant de vérifier si le même comportement est observé, ou s'il est spécifique au volume que nous avons choisi.

En étudiant T3, il sera possible de confirmer ou non la tendance avantageuse sur les lectures logiques qu'a

l'ETL face à l'architecture fédérée. L'étude permettra également de comprendre si cette tendance est globale, ou liée au type de requête demandée.

6.3.2 T3 - CTE avec fenêtrage + LAG

Pour la requête T3, on se concentre sur les lectures logiques pour appuyer l'hypothèse posée dans l'introduction, énonçant que les lectures seront significativement plus nombreuses pour les architectures fédérées pour les requêtes complexes incluant du fenêtrage + LAG.

L'étude permettra aussi de vérifier les tendances remarquées lors de l'étude de T2. La figure 6.5 illustre les mesures étudiées :

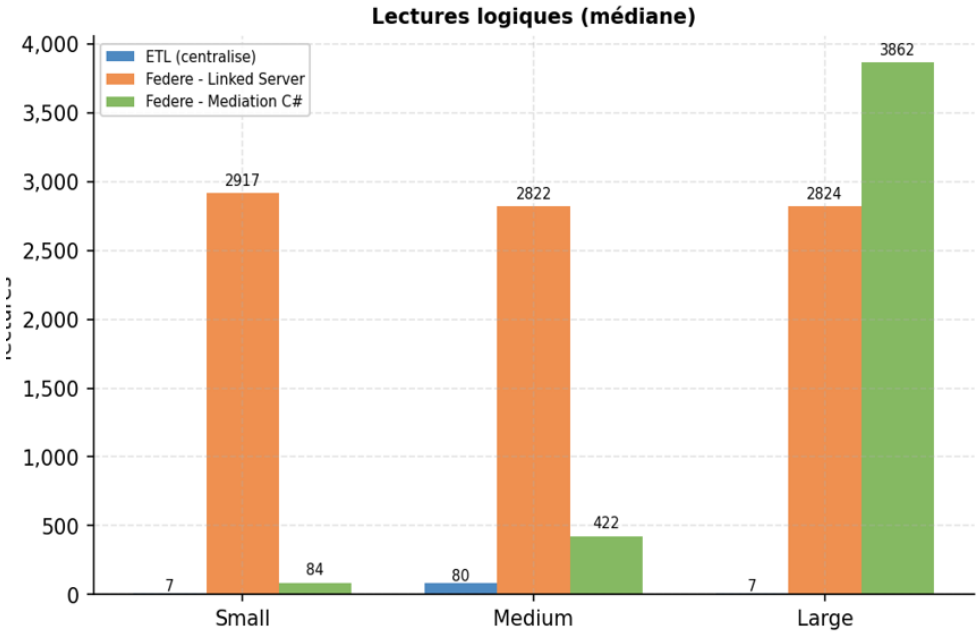

FIGURE 6.5 – Graphique des lectures logiques pour T3

On remarque des tendances distinctes pour les trois architectures.

L'ETL garde la tendance remarquée lors de T2, un nombre négligeable de lectures logiques, ceci grâce à la colocalisation des données ce qui permet d'exploiter les optimisations SQL Server au maximum.

La requête T3 porte sur une pièce fixe, et le générateur génère un nombre fixe de mesures par pièce, ce qui implique que même si l'on augmente en volume, le rapatriement des données devrait avoir un coût similaire pour chaque architecture étant donné que la requête récupérera dans tous les cas la même pièce.

La fédération LinkedServer montre des lectures logiques constantes sur les trois volumes, c'est en accord avec le comportement attendu mentionné dans le paragraphe précédent. Nous pouvons supposer que les légères variations entre chaque volume peuvent être dues à du bruit de mesures.

Dans le cas de la fédération par médiation cependant, on remarque que le coût de rapatriement semble proportionnel au volume.

Il est possible que ce soit dû au fonctionnement même de la médiation, cette dernière se base sur deux requêtes distinctes envoyées chacune sur une des bases dans le but de récupérer les données brutes à traiter ensuite du côté applicatif. Les données ne subissant aucun traitement côté SQL Server, il est normal que plus l'on ait de données dans les bases, plus l'on récupère de données brutes à traiter. Ce qui augmente le nombre de lectures logiques proportionnellement à la montée en volume. On peut en déduire que cette architecture pour ce type de requête spécifiquement serait particulièrement sous pression pour des volumes plus conséquents, comme on en trouverait typiquement en milieu industriel.

6.3.3 Synthèse T2 + T3

L'étude montre que la tendance dominante de l'ETL est globale sur les requêtes complexes. On verra plus tard lors de l'analyse de T6 le coût de l'ETL en ingestion batch, et cela nous permettra d'avoir une meilleure vue sur les avantages et inconvénients de cette architecture.

En ce qui concerne les architectures fédérées, on remarque que la fédération LinkedServer ne souffre de problème d'optimisation que lors de l'exécution de jointure, comme le démontre le scénario T2. Ainsi, on en déduit que pour les jointures spécifiquement, dans les volumes proches de celui utilisé pour le niveau medium, il y a un risque que SQL Server fasse un mauvais choix d'optimisation, sans que cela ne se répercute sur des requêtes comme T3.

La fédération par médiation, quant à elle, conserve sa tendance à croître proportionnellement au volume, néanmoins on remarque que la croissance est beaucoup plus prononcée dans le scénario T3 comparé à T2 où le phénomène est moindre. De ce fait, la solution pourrait sembler adaptée dans le cas d'opérations de jointures, tant que le volume reste modéré. Pour des requêtes qui montent en complexité comme T3, cependant, l'architecture devient trop coûteuse en termes de rapatriement de données, les volumes industriels étant souvent bien plus grands que les 3 millions de lignes de nos tests.

La conclusion importante de cette étude est que l'architecture ETL semble la plus optimale pour répondre à des besoins de requêtes complexes, l'architecture étant très stable sur les différents volumes et assurant un nombre de lectures logiques nécessaire minime.

La fédération LinkedServer montre de sérieuses limitations sur des cardinalités précises lors des opérations de jointures, et est certes constante sur des requêtes comme T3, mais reste moins performante que l'ETL.

La fédération par médiation C# montrera très vite des limitations sur des volumes importants comme rencontré dans des milieux industriels dû à la croissance proportionnelle de son coût de rapatriement des données.

6.4 T4 - Écriture unitaire

Le test T4 est un INSERT simple, il insère une mesure dans la table mesure, et récupère l'ID de la mesure ajoutée. Ce test vient montrer les premières différences architecturales en écriture, bien qu'elles soient pour l'instant très légères.

Pour ce faire, nous allons nous appuyer sur le graphique de comparaison entre le temps d'exécution end-to-end et le temps d'exécution serveur, illustré par la figure 6.6 :

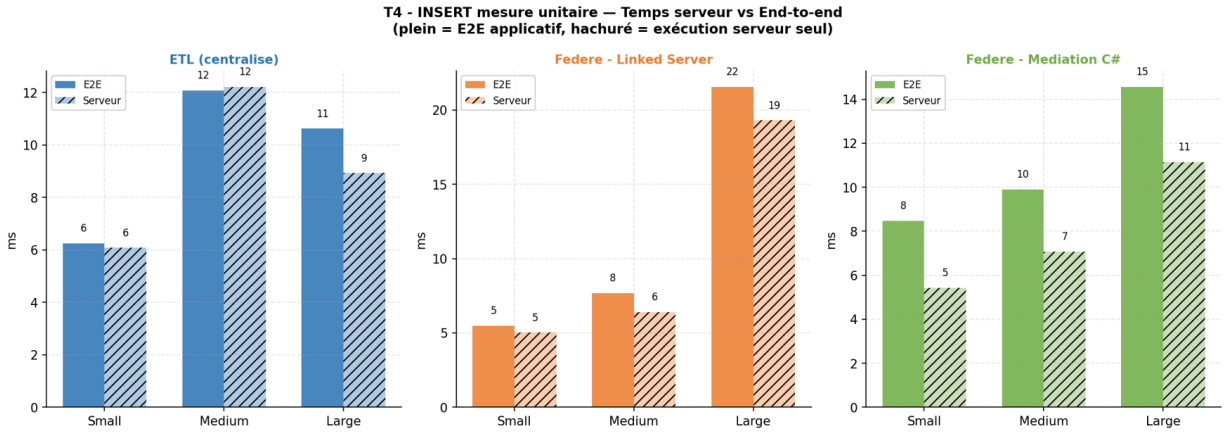

FIGURE 6.6 – Graphique du temps d'exécution end-to-end et serveur de T4

Grâce à ce graphique on peut observer une tendance intéressante sur chaque architecture.

L'ETL est encore une fois très stable, la seule différence apparaît sur le large, où le temps d'exécution end-to-end est très légèrement supérieur au temps d'exécution serveur, la différence est cependant négligeable et globalement l'ETL possède l'essentiel de sa charge côté SQL Server.

La fédération LinkedServer montre une certaine augmentation du temps d'exécution sur le volume large. Elle atteint les 22 ms end-to-end, et les 19 ms côté serveur, la différence reste faible, mais est quand même plus grande que les différences aperçues dans l'architecture ETL.

La charge reste majoritairement côté serveur, mais on observe une tendance qui nous indique de potentielles fortes augmentations sur des volumes plus grands. Le test T7 abordant l'insertion concurrente avec montée en charge devrait confirmer ou infirmer cette tendance.

Pour finir, la fédération par médiation C# met en valeur une différence légère mais notable entre le temps d'exécution end-to-end et serveur, ceci sur les trois volumes. On peut supposer que cette différence est due à un overhead applicatif, c'est-à-dire l'orchestration des deux requêtes distinctes ainsi que le traitement C# qui suit la récupération des données pour produire le résultat. La tendance globale semble être que l'on aura un overhead applicatif dû au fonctionnement même de l'architecture.

L'étude de T4 offre principalement un aperçu des tendances que l'on devrait observer de manière plus accentuée lors du test d'insertion avec montée en charge T7. Ce test permettra donc de valider ou d'invalider les différentes tendances que l'on a observées, tout en nous donnant une meilleure compréhension du comportement réel de chaque architecture dans des scénarios d'insertion.

6.5 T5 & T7 - Comportement sous charge

Les tests T5 et T7 ont pour but de mettre en valeur le comportement des différentes architectures sous charge. Ces derniers utilisent un fonctionnement commun, ils effectuent 10 tests consécutifs à plusieurs niveaux de connexions concurrentes, à savoir 10, 25, 50, 100 et 200 workers, soit 5 séries de tests. Pour chaque série, les requêtes seront exécutées une fois par worker, ce qui signifie que la montée en charge est évaluée purement via l'augmentation du nombre de connexions simultanées qui lance une requête.

La différence entre T5 et T7 apparaît sur le type de requête effectué.

T5 lance un burst de requêtes de sélection avec jointures, plus précisément, le test reprend la requête présente dans T2, et en lance autant qu'il y a de workers. Donc lors de la première série de tests, T5 lancera 10 requêtes de sélection en même temps, et ce à 10 reprises. Puis, il reproduira cela pour tous les niveaux définis.

T7 lance aussi un burst de requêtes, mais cette fois-ci ce sont des requêtes d'insertion simple. Et ce dernier reprend la requête utilisée pour T4, le reste du fonctionnement est identique à T5.

Le but de ces tests est d'évaluer l'évolution des métriques suivant la charge appliquée, pour ensuite récupérer les graphiques les plus révélateurs permettant d'obtenir un meilleur aperçu de l'architecture la plus performante sous contraintes.

6.5.1 T5 - SELECT avec jointures en rafale

L'étude de T5 est très révélatrice, elle permet d'observer des tendances sur plusieurs métriques telles que le débit en requêtes par seconde, le temps de complétion end-to-end et la consommation mémoire. Nous allons procéder point par point.

La figure 6.7 permet d'appuyer l'analyse des débits :

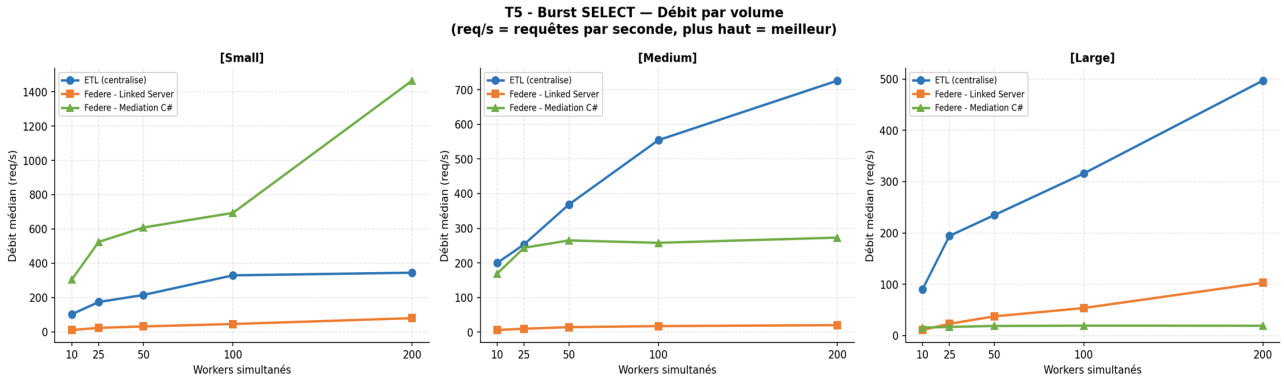

FIGURE 6.7 – Graphiques des débits de T5

Les graphiques ici révèlent une certaine domination de l'ETL sur les architectures fédérées dans les volumes plus grands.

Mais l'observation la plus intéressante est l'inversion ETL/Médiation C# entre le volume small et les volumes plus grands. Sur un volume petit, les données rapatriées par la fédération par médiation rentrent facilement en mémoire .NET. Étant donné que cette dernière fait le traitement des jointures côté applicatif, si toutes les données rentrent facilement en mémoire cela la rend d'autant plus rapide dans son exécution des requêtes. Lorsque l'on passe sur des volumes plus importants, les données ne tiennent plus toutes dans la mémoire, la médiation se trouve alors limitée par le mécanisme de GC, ce dernier suspendant tous les threads actifs (et donc des workers) pour libérer la mémoire qui n'est plus utilisée. C'est ce qui entraîne cette réduction du débit dans des volumes plus larges.

À l'inverse, l'ETL est limité sur small car sur de si petits volumes l'overhead SQL a un impact plus important sur le débit, ce qui limite l'ETL là où la médiation n'a pas ce problème.

Concernant la fédération LinkedServer, l'overhead TCP revient comme un facteur limitant, étant donné qu'on ne peut pas réduire cet overhead et qu'il est inhérent au transport inter-base, son impact est significatif même sur de petits volumes, ce qui limite grandement la fédération LinkedServer.

Ces observations se répercutent effectivement sur les temps de complétions respectifs de chaque architecture, comme le montre la figure 6.8 :

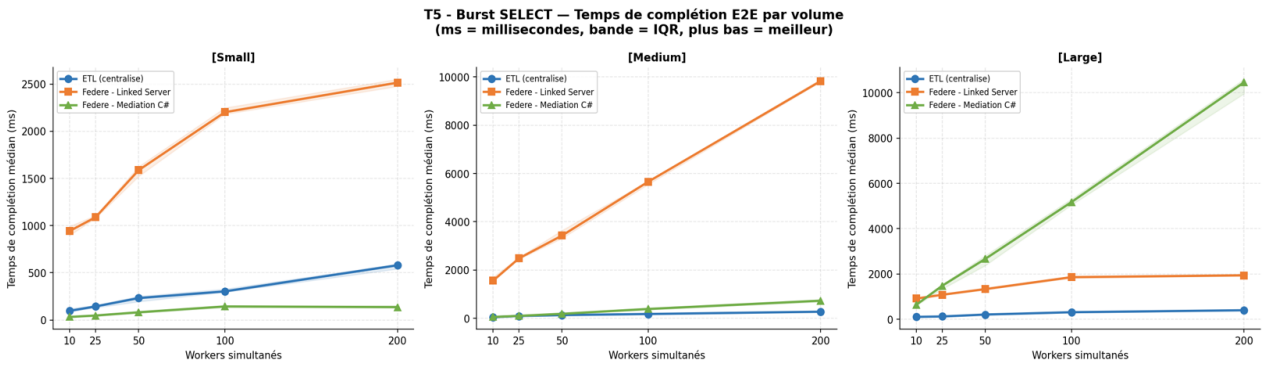

FIGURE 6.8 – Graphiques des temps de complétions de T5

Le temps de complétions met en lumière plusieurs faits intéressants.

Dans un premier temps, on confirme l'inversion ETL/Médiation C#, on remarque que lorsque l'ETL avait un débit inférieur à celui de la médiation (volume small), son temps de complétion est supérieur à celui de la médiation, indiquant qu'il a été moins rapide. Et cette tendance s'inverse à partir de medium tout comme le débit s'était inversé. On peut donc confirmer cette tendance pour nos tests.

Il est important de noter que la différence de débits remarquée plus tôt ($\tilde{4}80$ req/s), amplifie de manière significative la différence de temps de complétion observée sur le volume large ($\tilde{9}600$ ms), entre ETL et médiation C#.

On observe pour la fédération LinkedServer un temps de complétion élevé sur les 3 volumes. Sur small, ce temps de complétion est sûrement dû à l'overhead TCP que l'on a déjà mentionné plusieurs fois, étant inhérent au fonctionnement de LinkedServer son impact est significatif et ce sur plusieurs métriques comme on le voit maintenant. En valeur absolue, le volume large possède un temps de complétion similaire à small, mais il est "compressé" à cause de l'échelle de l'axe Y qui est dictée par le temps de complétion de la médiation C# qui explose sur ce volume.

Le plus remarquable pour LinkedServer reste le volume medium. On rappelle que T5 est un lancement en parallèle d'une multitude de requêtes T2, or on a vu que le plan d'exécution choisi pour LinkedServer dans cette cardinalité pour la requête T2 est non optimal, et cela se répercute sur les performances de T5. Si on combine le mauvais plan d'exécution à l'overhead TCP, et que l'on lance jusqu'à 200 requêtes simultanées dans ces conditions, le temps de complétion explose, ce qui explique les mauvaises performances de LinkedServer sur ce volume.

Enfin, la médiation C# montre une augmentation progressive mais significative au fil des volumes, la plus significative étant sur le volume large. On remarque aussi que son IQR est nettement plus visible que les autres architectures.

Ces deux comportements sont dus au fonctionnement même de la médiation, et notamment du GC. Comme mentionné précédemment, le GC cause une perte de débit ce qui se traduit par la suite par une augmentation du temps de complétion. L'IQR ici nous illustre un second problème lié au GC, l'imprévisibilité. En effet, on ne peut pas prévoir quand le GC va décider de se mettre en marche, ce qui fait que sur plusieurs tests, ce dernier ne s'activera pas au même moment et pas à la même fréquence, plus d'activations vont causer une augmentation du temps de complétion, et inversement s'il s'active moins souvent le temps de complétion sera meilleur. La médiation souffre donc de deux gros problèmes, un problème de performance et de constance dans ses résultats.

Le graphique de la consommation mémoire .NET, illustré par la figure 6.9, vient confirmer et expliquer ces comportements :

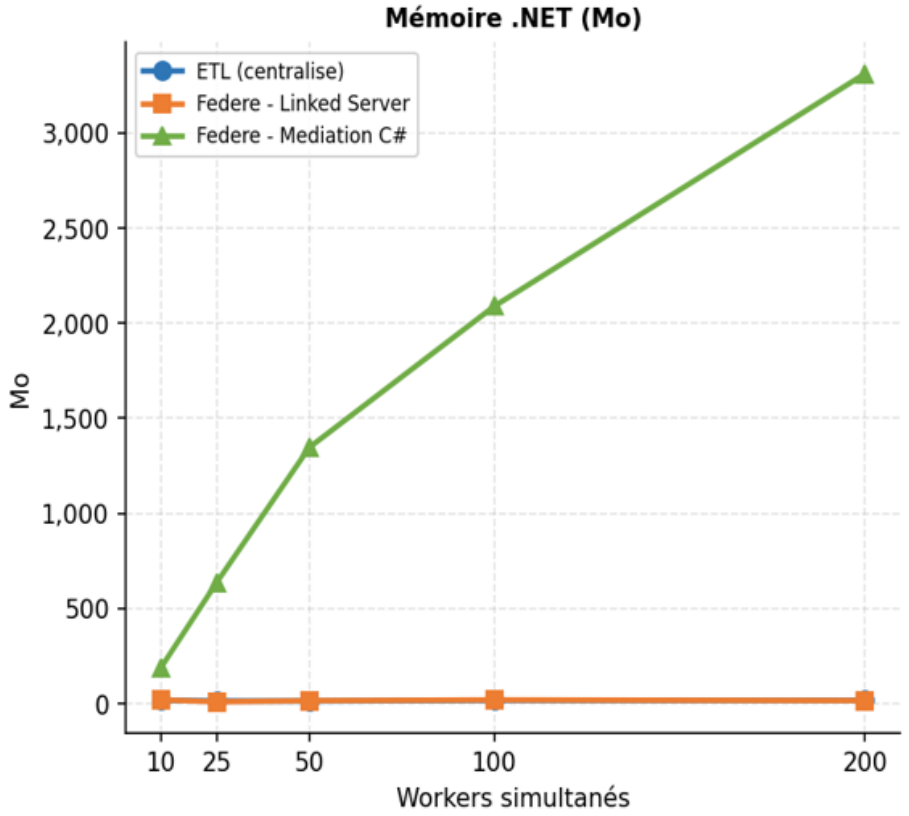

FIGURE 6.9 – Graphique de la consommation mémoire .NET de T5

On exploite seulement le graphique du volume large car c'est sur ce volume qu'il est le plus pertinent. On remarque notamment que la mémoire .NET requise par la médiation croît massivement avec le nombre de workers, on passe de $\tilde{2}00$ Mo pour 10 workers à $\tilde{3}500$ Mo requis pour 200 workers. Cette augmentation en mémoire utilisée est directement corrélée avec l'augmentation du temps d'exécution, on remarque que la courbe vue précédemment subit une évolution similaire à celle de la mémoire présentée ci-dessus. De plus on remarque un second phénomène lié à la mémoire. La courbe précédente possède un IQR qui augmente avec le nombre de workers, or on a supposé que l'IQR traduisait l'inconstance des activations du GC et que ce dernier était un problème à cause de la mémoire. La courbe de mémoire étudiée ici semble corroborer cette hypothèse.

Par ailleurs, on remarque que l'ETL et la fédération LinkedServer ont une consommation de mémoire négligeable et stable, il y a donc une différence architecturale marquante entre la médiation C# et les deux autres architectures. La médiation est la seule à souffrir de pression mémoire du côté applicatif.

L'étude de la requête T7 va permettre d'observer les performances des architectures en insertion, ce qui permettra enfin de croiser les observations de T5 et T7 et d'en dégager des conclusions plus solides sur les architectures.

6.6 T7 - INSERT simple en rafale

Tout comme pour T5, étudions d'abord les graphiques de débit illustrés par la figure 6.10 :

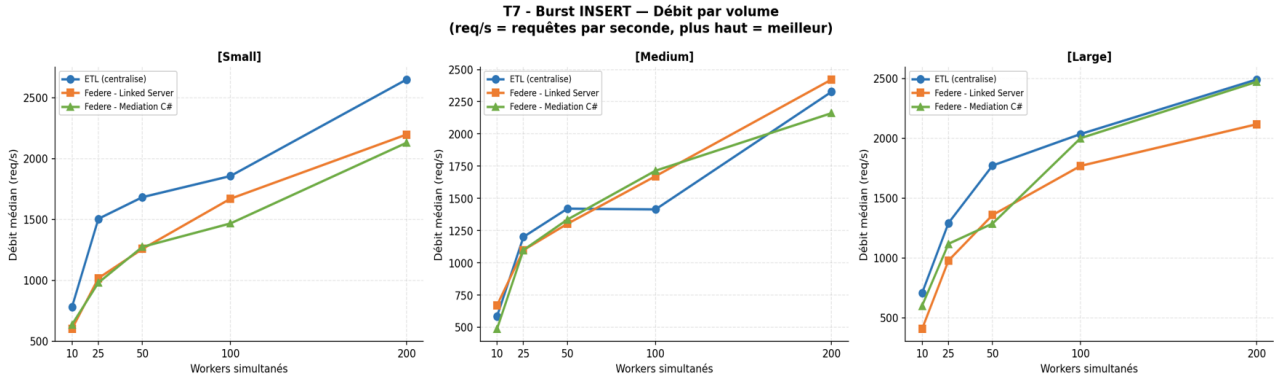

FIGURE 6.10 – Graphiques des débits de T7

Les graphiques montrent que la tendance dominante de l'ETL sur les sélections est conservée sur les insertions, avec cependant une légère baisse de performances sur le volume medium.

On peut supposer que cette légère baisse de performance pourrait être due à de la contention côté SQL Server. En effet, sur medium, les pages et index disponibles sont moins nombreux et dispersés, les workers sont donc plus susceptibles de cibler les mêmes pages d'index ou pages de données sur le volume medium par rapport au volume large. Cela implique plus de risque de contention et explique pourquoi medium marque une légère baisse de performance côté ETL, le rapport nombre de mesures insérées/volume doit être proche du seuil où la contention montre un léger impact.

La tendance globale confirme tout de même une dominance relative de l'architecture ETL bien que les écarts soient moins marqués que lors de l'étude des débits de T5.

La tendance globale pour la fédération est aussi confirmée, elle reste inférieure à l'ETL, cependant on remarque que les performances d'insertions pour ces dernières sont quand même relativement plus proches de l'ETL qu'en sélection.

Étudions maintenant les temps de complétions grâce à la figure 6.11 et voyons s'il y a corrélation avec les observations des débits :

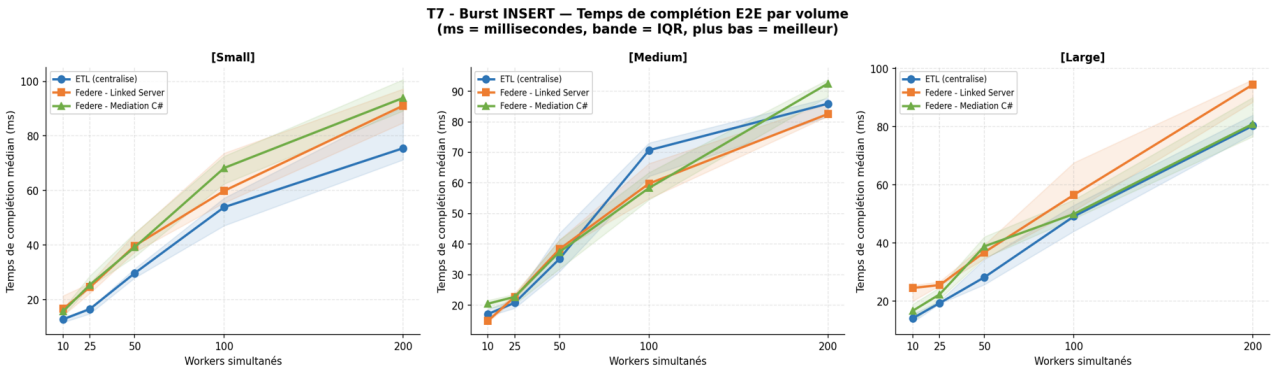

FIGURE 6.11 – Graphiques des temps de complétion end-to-end de T7

Les graphiques présentés pour les temps de complétion sont révélateurs de deux points majeurs.

Dans un premier temps, ils valident les observations faites au niveau des débits de T7, on a là aussi une dominance de l'ETL sur les architectures fédérées, on y retrouve aussi la légère baisse de performance sur le volume medium. Le rapprochement relatif entre ETL et architecture fédérée est aussi illustré dans ces graphiques, ce qui renforce nos constats précédents.

On remarque dans un second temps un changement significatif comparé à T5, les IQR sont de manière

globale bien plus importants. L'IQR est un indicateur de la stabilité de la mesure, ici il nous montre donc qu'en insertions, les architectures ont des performances significativement moins stables qu'en sélection.

Ceci est dû à la nature beaucoup plus variable des insertions, ces dernières impliquant des verrous, des mises à jour d'index et des écritures sur disque, contrairement aux requêtes de sélection qui n'impliquent pas ces opérations.

Enfin, l'analyse de la mémoire SQL montre un comportement intéressant au niveau de l'ETL, ce que nous montre la figure 6.12 :

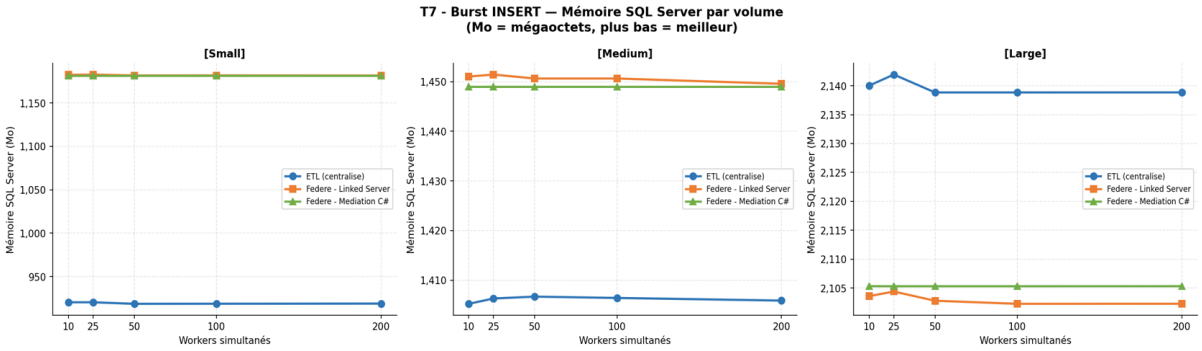

FIGURE 6.12 – Graphiques de la mémoire SQL de T7

On remarque que pour les volumes small et medium, ce sont les architectures fédérées qui demandent le plus de mémoire SQL, l'ETL reste le moins coûteux. Cependant, sur le volume large l'ETL devient celui qui consomme le plus de mémoire.

La différence de consommation mémoire est plus marquée sur small ($\tilde{2}40$ Mo), en faveur de l'ETL. On remarque cependant que cette tendance s'estompe sur des volumes croissants et éventuellement s'inverse.

Cette inversion est cependant difficilement explicable sans métriques supplémentaires.

Les grands volumes étant ceux rencontrés dans un milieu industriel, le fait que la différence s'estompe quand on s'en rapproche la rend négligeable au regard de nos observations. Il serait néanmoins intéressant de tester des volumes plus significatifs pour observer si l'écart en faveur des fédérations reste négligeable ou augmente.

T7 nous permet donc d'observer que les tendances globales de l'ETL et des architectures fédérées sont conservées en insertion. La nature des insertions rend cependant ces tendances plus instables, comme le démontre l'IQR observé sur les temps de complétion.

Il nous permet aussi de constater une observation intéressante sur la mémoire SQL Server, sans nous permettre de conclusions pertinentes au regard des métriques disponibles.

À noter, la requête d'insertion étant plus simple que la requête de sélection utilisée en T5, les résultats de T7 sont par conséquent moins discriminants entre les architectures, ce qui explique que les écarts entre les débits et les temps de complétion soient moins importants que lors de l'étude des mêmes métriques dans T5.

Dans l'analyse de T6, nous allons observer le coût d'ingestion de l'ETL pour mieux comprendre quel est l'inconvénient de cette architecture dans des environnements où les données sources évoluent rapidement. Le test T6 est exclusif à l'ETL, les architectures fédérées n'ayant pas de système d'ingestion car elles accèdent directement aux données sources.

6.7 T6 - Ingestion batch

Le test T6 permet de mesurer le temps d'ingestion batch de l'ETL sur les trois volumes. Pour ce faire il effectue la récupération des lignes de mesures de la base de production (Bench_Production) vers la base principale centralisée (Bench_Centralized) grâce à SqlBulkCopy. On mesure le débit d'ingestion en lignes par seconde ce qui nous permet de comparer sur différents volumes à quelle vitesse l'ETL arrive à récupérer les données et la durée totale requise par cette opération.

Notre étude se basera sur la figure 6.13 :

Volume	Duree mediane (s)	IQR (s)	Debit (lignes/s)	Lignes copiees
Small	0,7	0,0	29 135	20 000
Medium	9,2	0,3	27 200	250 000
Large	113,1	0,8	26 523	3 000 000

FIGURE 6.13 – Tableau de l'ingestion ETL durant T6

Ce tableau nous montre que l'ingestion se fait de manière stable dans l'ensemble, on a une légère baisse de débit entre les différents volumes que l'on peut attribuer à la durée des tests. En effet, la mesure étant la médiane, un test plus long va produire plus de mesures et retourner une médiane plus significative de la vitesse de transfert des données.

Le temps de complétion croît proportionnellement au volume de la base, atteignant Ĩ13s pour le volume large, soit 1 min 53.

L'IQR est négligeable, n'atteignant pas 1 seconde même sur le volume large, cela montre une très bonne stabilité dans l'ingestion des données.

La figure 6.14 permet une meilleure visualisation de l'évolution des métriques en fonction du volume :

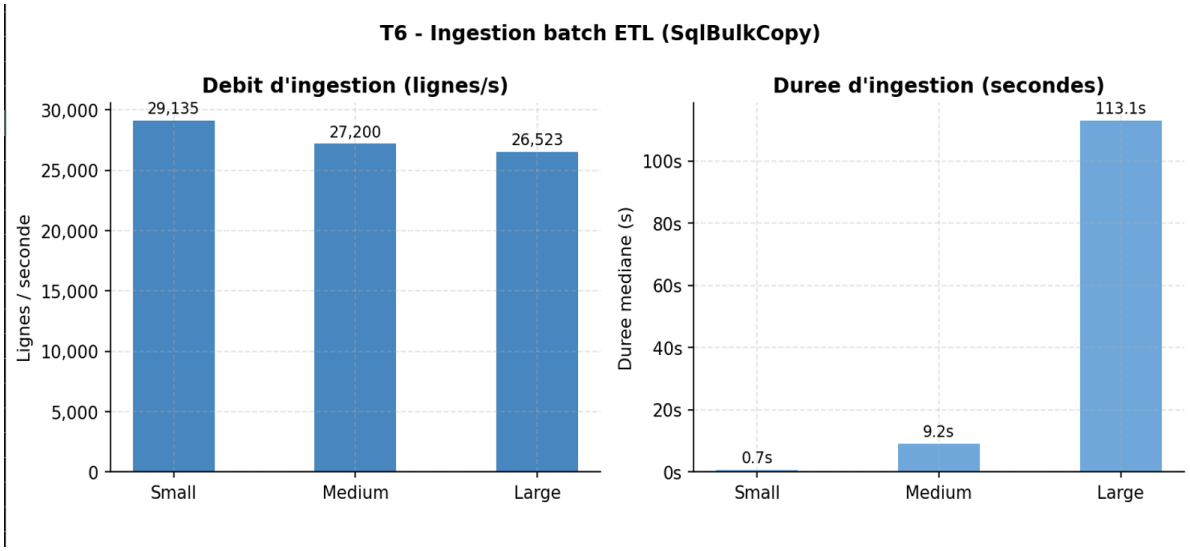

FIGURE 6.14 – Graphique de l'ingestion ETL durant T6

Les graphiques confirment bien que le temps d'ingestion augmente majoritairement en fonction du volume et non pas à cause de la réduction du débit.

Globalement, T6 nous montre que l'ingestion de l'ETL est rapide et stable, c'est ce que l'on cherche à obtenir pour limiter l'impact de l'ingestion sur la fraîcheur des données. Cela reste néanmoins le facteur limitant de l'architecture, la rendant moins intéressante dans les systèmes en temps réel.

6.8 Synthèse de l'étude

L'étude complète de chaque test nous permet d'identifier les avantages et inconvénients de chaque architecture.

L'ETL est dominant sur les requêtes complexes, comme le montrent les tests T2 et T3, de plus il se comporte très bien sous charge avec une grande stabilité en sélection comme le montre T5, et une moins bonne stabilité en insertion, caractéristique qui est partagée par toutes les architectures, comme le montre T7, ceci étant inhérent aux insertions elles-mêmes. T6 met en valeur le réel coût de l'ETL, l'ingestion, qui limite donc la fraîcheur des données et rend l'architecture moins pertinente dans des systèmes en temps réel.

La fédération LinkedServer montre des limitations sur les jointures, notamment sur le volume medium comme le montre T2. Les performances globales restent inférieures à celles de l'ETL et de la fédération par médiation C# dans la majorité des scénarios.

Enfin la fédération par médiation C# montre des limites liées aux grands volumes dues à la mémoire nécessaire pour répondre aux requêtes, comme le montre T5. La solution devient donc particulièrement limitée pour répondre à des besoins sur des bases de tailles significatives. Sur de petites bases la solution reste compétitive, produisant des résultats similaires voire meilleurs que l'ETL dans le cas de T5. Mais globalement la médiation C# reste elle aussi inférieure à l'ETL

L'avantage principal des fédérations est qu'elles n'ont pas de coût d'ingestion, elles récupèrent les données en temps réel ce qui les rend préférables dans des systèmes en temps réel.

En conclusion, l'architecture ETL s'impose comme la plus performante pour des systèmes nécessitant l'exécution de requêtes complexes sur de grands volumes de données, sans contrainte de fraîcheur de données.

La fédération C# reste compétitive pour l'exécution de requêtes simples sur des volumes plus petits et nécessitant un accès aux données en temps réel.

Enfin, la fédération LinkedServer ne présente pas d'avantage significatif dans les tests effectués, et reste globalement moins performante que les deux autres architectures étudiées. Elle reste une solution potentielle dans le cas où un ETL ou une médiation applicative n'est pas possible.

À noter, les conclusions retenues de ces études sont propres à l'environnement de test utilisé. Étant une machine locale, certaines valeurs absolues peuvent être sujettes à des bruits de mesures plus ou moins importants. Il est donc important de surtout prendre en compte les tendances observées par rapport aux valeurs absolues. Des tests sur des machines dédiées pourraient venir confirmer ou contredire les conclusions de cette étude.

Chapitre 7

Conclusion générale et ouverture

Ce travail avait pour but d'exposer les différentes solutions de centralisation des données et d'étudier les plus pertinentes à mettre en place dans le cas de l'architecture de l'entreprise.

Après avoir expliqué l'architecture actuelle et observé les différentes solutions courantes dans les entreprises, nous avons présenté l'architecture à mettre en place pour les tests, laquelle reflète l'architecture multi-base industrielle présente en interne, puis avons expliqué le protocole qui a été utilisé pour les tests.

Enfin, l'étude des différents scénarios retenus nous a permis de mieux comprendre quelle architecture répond aux besoins de l'entreprise.

Le travail souligne l'utilité des ETL dans une architecture dédiée à l'analytique, tandis que les fédérations se conforment mieux à des architectures de production industrielle.

À l'avenir, des tests plus complets pourraient être menés sur des machines dédiées avec des volumes industriels réels, et des requêtes plus complexes, notamment en insertion. Cela permettrait aussi d'étudier l'impact d'un réseau réel entre les bases et de le comparer à l'impact qu'a eu le loopback sur nos conclusions. En effet, l'impact que le loopback a eu sur nos résultats sous-estime sûrement l'impact du réseau réel, ce qui pénaliserait encore davantage le LinkedServer notamment.

Les résultats pourraient ainsi venir compléter cette étude et permettre le choix et le déploiement de la solution la plus adaptée aux besoins réels de l'entreprise.

Bibliographie :

— **Talend**. *Guide sur les ETL*. https://www.talend.com/fr/resources/guide-etl/

— **Wikipedia**. *Jointure (informatique)*. https://fr.wikipedia.org/wiki/Jointure_(informatique)

— **Oracle**. *Définition base de données distribuée*. https://www.oracle.com/fr/cloud/definition-base-donnee-distr

— **Wikipedia**. *Multibases*. https://fr.wikipedia.org/wiki/Multibases

— **IMT Atlantique**. *Jointures SQL*. https://formations.imt-atlantique.fr/bd_ihm/fr/sql/jointures/

— **AWS**. *Techniques d'optimisation de requêtes*. https://docs.aws.amazon.com/fr_fr/athena/latest/ug/performance-tuning-query-optimization-techniques.html

— **Aras**. *Centralised Data Management*. https://aras.com/fr-fr/glossary/centralized-data-management

— **Microsoft Azure**. *Qu'est-ce qu'un Data Warehouse ?*. https://azure.microsoft.com/fr-fr/resources/cloud-computing-dictionary/what-is-a-data-warehouse

— **Salesforce**. *Définition Data Warehouse*. https://www.salesforce.com/fr/resources/definition/data-warehouse

— **HubSpot**. *Centralisation de données*. https://blog.hubspot.fr/sales/centralisation-de-donnees

— **Solutions Business Intelligence**. *Data Warehouse : quelle architecture ?*. https://solutions-business-intellige fr/data-warehouse-quelle-architecture/

— **Skiils**. *Data Mesh vs Data Warehouse*. https://www.skiils.com/blog/data-mesh-vs-data-warehouse

— **RiseUp**. *Centralisation des données*. https://www.riseup.ai/fr/blog/centralisation-des-donnees

Note sur l'IA

L'outil IA Claude (Sonnet 4.6) a été utilisé dans ce travail pour le développement complet du benchmark et du script permettant de récupérer les graphiques.

L'entièreté du code et des résultats, même les tableaux et graphiques non exploités sont trouvables sur GitHub avec ce lien : https://github.com/trystancesi/Benchmark-ETL-Federation.git

Le reste de son utilisation se limite à de l'aide à la compréhension d'idées et à de la reformulation occasionnelle. Tous les textes ont été relus et reformulés dans un but d'appropriation du travail.